IUBAT— INTERNATIONAL UNIVERSITY OF BUSINESS AGRICULTURE AND TECHNOLOGY

Lab Report

Course Code: CSC 434

Course Title: Database Management System Lab

Title of Work: Create project on Blood Bank Management System.

Submitted To

Name: Faniyam Maria Mansia

Lecturer,

Department of Computer Science and Engineering.

Submitted By

Name: Ehetisum Sharif

ID: 23103380

Program: BCSE

Section: G

Date of submission: 28 January 2025

Table of Contents

Blood Bank Management System

Preface

Programming is a skill best learned by doing. In a controlled, hands-on setting, we will gain practical experience in writing, editing, compiling and executing programs. This approach helps reinforce the theoretical concepts learned in class while also allowing us to tackle real-world challenges. By actively engaging with code, we can enhance our problem-solving abilities, understand the nuances of debugging and improve the quality of our programs. Through continuous practice our programming skills will steadily improve, enabling us to develop robust systems, such as the Blood Bank Management System (BBMS) and address real-world issues effectively.

Topic

HTML, CSS, PHP and MySQL, Query Processing

Objective

The goal of Blood Bank Management System (BBMS) is to use modern database technologies to manage blood donations, donor and recipient data and blood stock efficiently.

1. **Database Management:** Use relational databases (MySQL, PostgreSQL) or NoSQL (MongoDB) to store data, ensuring scalability and data integrity.
2. **Architecture:** Design the database with an Entity-Relationship (ER) model and ensure ACID properties for transaction consistency.
3. **Web Application:** Develop a web-based system using HTML, CSS, and PHP (or other backend languages) for CRUD operations, user authentication and database interaction.

This approach ensures efficient management, smooth operations and data integrity for the BBMS.

Theory

Evolution of Database Management Systems (DBMS)

For the BBMS project, it's crucial to select a database system that supports large-scale data management while providing scalability and flexibility. Given the evolution of Database Management Systems (DBMS) here how different systems align with your project's requirements:

1. **Early File Systems**:
    - These systems stored data in flat files and faced issues like data redundancy, inconsistency and difficulty in querying. While simple, they are not suitable for

handling the complex, high-volume data you require for donors, recipients and blood stocks.

2. **Relational Databases**:
    - Relational databases like MySQL are an excellent choice for the BBMS project. They offer well-structured tables, which can be highly beneficial for storing structured data (like donor information and blood stocks) and ensuring data consistency with features such as ACID (Atomicity, Consistency, Isolation, Durability) properties.
    - SQL queries allow for efficient data retrieval, management and reporting. Relational databases also support normalization, helping eliminate redundancy and improve data integrity, which is critical for your application.
    - Scalability options such as replication, partitioning and clustering can handle growing data demands.
3. **Object-Oriented Databases**:
    - While object-oriented databases, such as ObjectDB, offer a more natural way to store data that directly reflects objects in object-oriented programming (OOP) they might be overkill for your needs unless complex relationships between objects (like entities having many-to-many relationships) are present.
    - These databases could be considered if you expect to integrate the DBMS closely with object-oriented application code.
4. **NoSQL Databases**:
    - NoSQL databases like MongoDB and Cassandra are designed for flexibility and scalability, especially in handling unstructured or semi-structured data. They are ideal for applications with large datasets and when high write throughput or real-time data access is necessary.
    - While powerful in many use cases, NoSQL might not be as well-suited for the BBMS project, where data consistency and structured query capabilities (such as those offered by relational databases) are crucial. However, NoSQL could be considered if your data structure evolves to include more unstructured data or if the system grows in complexity.
5. **Emerging Trends: Cloud Databases**:
    - Cloud databases offer high availability, scalability, and real-time data processing. If the BBMS project requires flexibility in terms of infrastructure or real-time data access, cloud-based solutions like Amazon RDS (which supports MySQL, PostgreSQL, etc.) or hybrid models that combine the benefits of both relational and NoSQL databases (e.g., Google Cloud Spanner or Azure Cosmos DB) could be the way to go.
    - Cloud solutions also reduce the overhead of maintaining physical infrastructure and offer flexibility for future scaling.

So, a relational database like MySQL would be ideal for managing donor, recipient and blood stock information, ensuring data integrity and supporting complex queries. As the system grows, we might explore cloud-based or hybrid solutions to meet the increasing scalability and availability demands.

Entity Relationship Modelling and Design (Conceptual Modelling)

Entity Relationship (ER) Modelling plays a vital role in designing the Blood Bank Management System (BBMS) by providing a clear structure for the data and the relationships between key entities such as donors, recipients, blood units and administrators. This conceptual model ensures that all necessary data points are captured accurately and that the relationships between entities are represented clearly, supporting the overall functionality of the system.

Key Components of ER Modelling for BBMS:

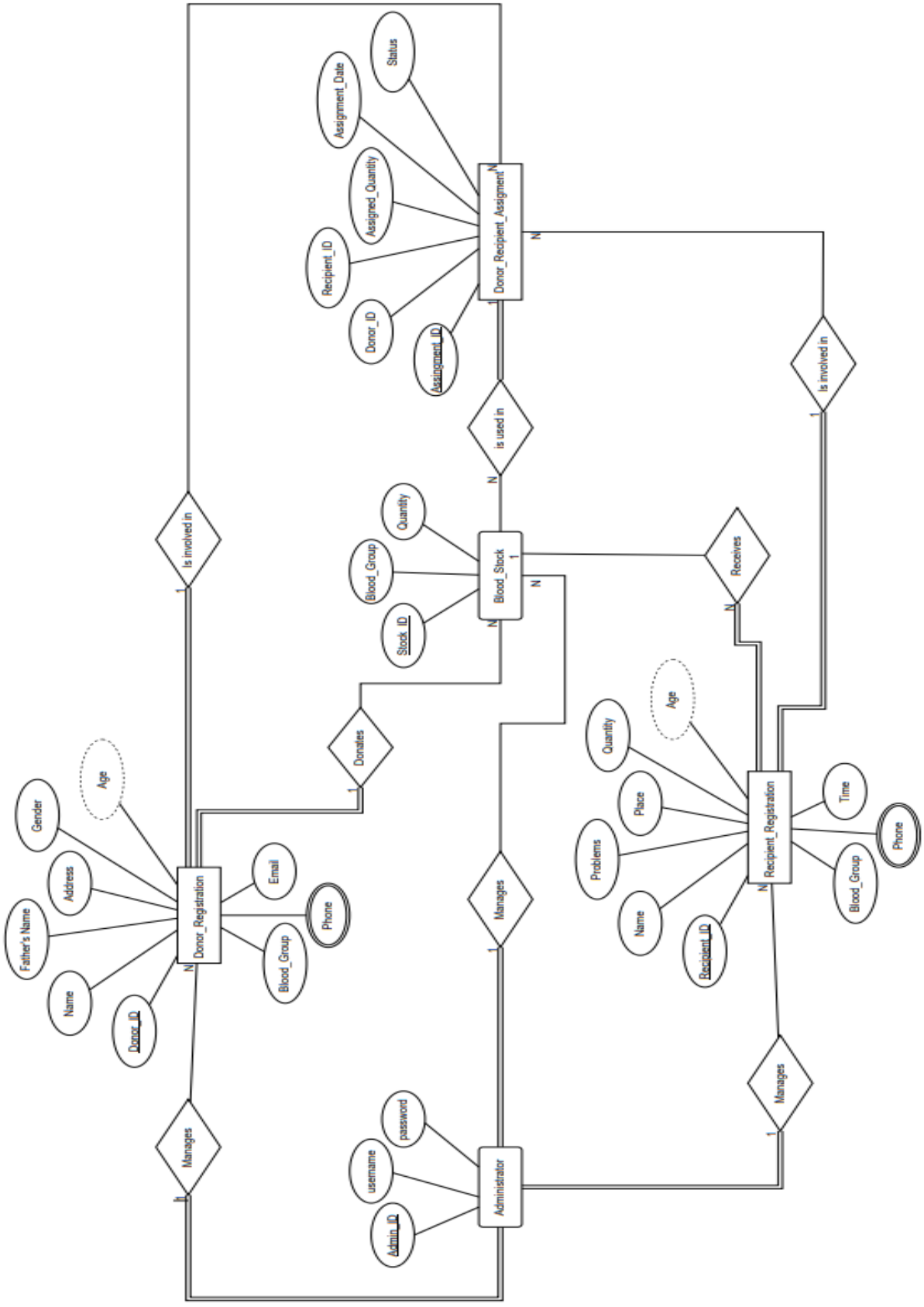
1. **Entities**: Entities represent key objects or concepts within the system. In the context of the BBMS, the following entities are essential:
    - **Admin**: The administrator responsible for managing the BBMS system, including overseeing blood stock and registrations.
    - **Blood Stock**: Represents the blood units available for donation or distribution.
    - **Donor Registration**: Represents individuals who donate blood, containing personal and contact information.
    - **Recipient Registration**: Represents individuals in need of blood, capturing relevant medical and contact information.
    - **Donor-Recipient Assignment**: Tracks the assignment of blood donations to specific recipients.
2. **Attributes**: Each entity has specific attributes that define its characteristics:
    - **Admin**: id, u_name, password.
    - **Blood Stock**: id, bgroup, quantity.
    - **Donor Registration**: id, name, fname, address, gender, age, bgroup, email, pno.
    - **Recipient Registration**: id, name, age, problems, bgroup, quantity, place, time, pno.
    - **Donor-Recipient Assignment**: id, donor_id, recipient_id, assigned_quantity, assignment_date, status.
3. **Relationships**: Relationships between entities define how they interact within the system:
    - **Admin → Blood Stock / Donor / Recipient**: An administrator manages all aspects of the BBMS, overseeing blood stocks, donor registrations and recipient registrations.
    - **Donor → Blood Stock**: A donor donates blood, which adds to the available blood stock. This relationship is one-to-many because a donor can contribute multiple times to the blood stock.
    - **Blood Stock → Recipient**: A recipient can receive blood from the available stock. This relationship is many-to-one, where multiple recipients can be assigned blood from the same stock.
    - **Donor → Donor-Recipient Assignment**: A donor can be linked to multiple recipient assignments. This is a one-to-many relationship, as one donor may donate to multiple recipients.

- **Recipient → Donor-Recipient Assignment**: Each recipient can be linked to multiple donor assignments, as they may receive blood from different donors. This is also a one-to-many relationship.

4. **Cardinality**: Cardinality defines how many instances of one entity can be associated with another:
   - **Donor → Blood Stock**: One-to-many. A donor can donate multiple times to the blood stock.
   - **Blood Stock → Recipient**: Many-to-one. A blood stock can be assigned to multiple recipients, but each blood unit is assigned to one recipient at a time.
   - **Donor → Donor-Recipient Assignment**: One-to-many. A donor can have multiple assignments to different recipients.
   - **Recipient → Donor-Recipient Assignment**: One-to-many. A recipient may be linked to multiple donor assignments based on their blood needs.
5. **Participation**: Participation specifies whether entities must participate in a relationship:
   - **Donor**: The participation of donors is mandatory because they must register and donate blood to the system in order to contribute to the blood stock.
   - **Recipient**: The participation of recipients is optional. Not all recipients may need blood at the same time, and some may not require donations.
   - **Blood Stock**: The participation of blood stock is partial because some blood units may remain in storage without being assigned to a recipient.
   - **Admin**: The participation of the admin is total as they must manage the BBMS and oversee all operations.

The ER diagram for the Blood Bank Management System (BBMS) is essential in organizing and structuring the data in a way that ensures efficient processing and data integrity. By clearly defining the entities, their attributes and relationships, ER modelling provides a foundation for a robust database system that supports the functionality and operations of the BBMS. The relationships, cardinality and participation rules ensure that data is accurately captured, processed and maintained throughout the system.

ER diagram:


Donor_Registration
Donor_ID
Name
Father's Name
Address
Gender
Age
Email
Phone
Blood_Group
Is involved in
Donates
Blood_Stock
Stock_ID
Blood_Group
Quantity
Donor_Recipient_Assigment
Assignment_ID
Donor_ID
Recipient_ID
Asaigned_Quantity
Assignment_Date
Status
is used in
Is involved in
Receives
Recipient_Registration
Recipient_ID
Name
Problems
Place
Quantity
Age
Blood_Group
Time
Phone
Manages
Manages
Manages
Administrator
Admin_ID
username
password

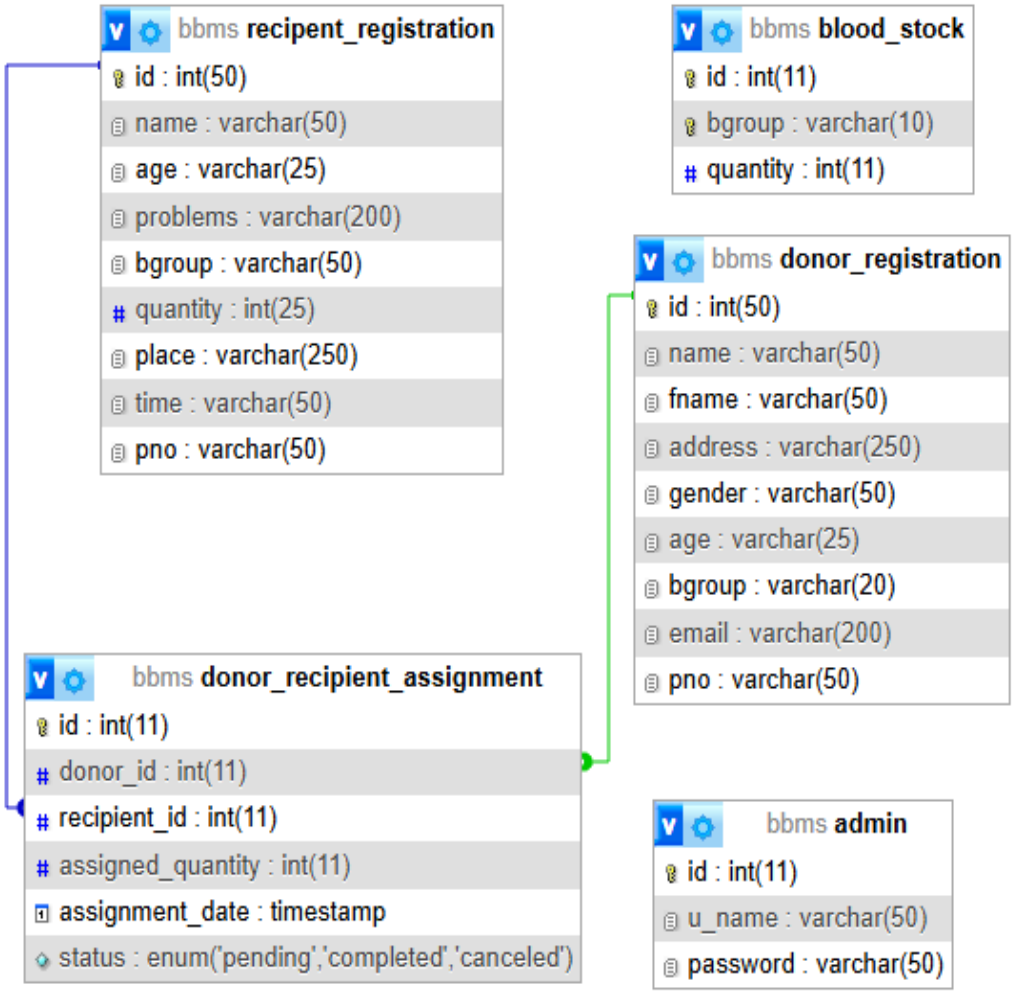
Relational Data Model and Relational Algebra (Logical Modelling)

The relational data model and relational algebra are essential concepts for working with databases, especially in systems like BBMS (Blood Bank Management System), where efficient data retrieval and manipulation are crucial.

1. **Relational Data Model**:
    - In this model, data is organized into tables (relations), with each table consisting of tuples (rows) and attributes (columns).
    - The relational model allows for structured data organization, where each table represents a specific entity (e.g., blood donors, recipients, blood types).
    - Primary keys uniquely identify each tuple, while foreign keys create relationships between different tables (e.g., linking a blood donation record with a specific donor).
2. **Relational Algebra**:
    - This set of operations is used to query and manipulate relational data. These operations can be performed using SQL (Structured Query Language), which is the practical implementation of relational algebra in relational database systems.
    - Key operations include:
        - **Selection** (σ): This operation extracts rows from a table that satisfy a specified condition. For example, to find all available blood units of a specific blood group, we could select the rows where the blood group matches the required one.
        - **Projection** (π): This operation extracts specific columns from a table. For instance, projecting the blood unit ID and its availability status without retrieving other unnecessary details.
        - **Union** ($\cup$), **Difference** ($-$), and **Intersection** ($\cap$): These operations manipulate sets of tuples from different tables. For example, union can combine two sets of blood donation records from different donors.
        - **Join** ($\bowtie$): This operation combines data from two or more tables based on a common attribute, such as joining a table of blood donations with a table of blood donors based on a donor ID.

In the case of the BBMS, these operations allow efficient data querying, such as checking for available blood units by blood group, identifying which donors have donated recently or tracking the history of blood donations and transfusions.

Relation Schema Diagram:



Structured Query Language (Physical Modelling)

Structured Query Language (SQL) plays a crucial role in implementing the physical model of the BBMS (Blood Bank Management System) database by enabling the creation, manipulation and control of data stored in a relational database. Here's a breakdown of the three main SQL categories as they apply to the BBMS:

1. **Data Definition Language (DDL)**:
    - SQL DDL commands are used to define the structure of the BBMS database. This includes:
        - **CREATE**: To create tables such as donor_registration, recipient_registration, blood_stock, etc.
        - **ALTER**: To modify existing tables, such as adding new columns (e.g., phone for donors).
        - **DROP**: To delete tables or other database objects when they are no longer needed.

2. **Data Manipulation Language (DML)**:
    - DML commands allow interaction with the data within the BBMS database:
        - **SELECT**: To retrieve specific data, such as viewing blood stock levels or donor information.
        - **INSERT**: To add new records, such as inserting a new donor into the database.
        - **UPDATE**: To modify existing records, for example, updating a donor's contact information or changing the status of blood bags.
        - **DELETE**: To remove data, like deleting outdated donation records or expired blood stock.
3. **Data Control Language (DCL)**:
    - DCL commands help manage access to the BBMS database, ensuring only authorized users can perform sensitive operations:
        - **GRANT**: To provide specific privileges to users, such as allowing the system administrator to grant access to medical staff for accessing donor information.
        - **REVOKE**: To remove previously granted privileges from users, maintaining security and preventing unauthorized access.

By using these SQL commands, the BBMS can securely store and manipulate data, ensuring the efficient management of blood donations, inventory and donor information while providing proper access control to ensure data security.

Transaction Processing and Concurrency Control

In a Blood Bank Management System (BBMS), both transaction processing and concurrency control play a critical role in maintaining the integrity and reliability of the system. Here's a deeper explanation of how each of these concept's functions within the system:

Transaction Processing:

Transaction processing in BBMS ensures that all operations such as adding new donors, issuing blood to recipients, or updating inventory are either completed fully or not executed at all in case of an error. This is achieved through the ACID properties:

1. **Atomicity**: Guarantees that a transaction is all-or-nothing. For example, if blood is being issued to a recipient, the entire transaction (updating inventory, logging the transaction and notifying both parties) will either succeed or fail together. If there's an error (e.g., inventory update fails), the entire process is rolled back, avoiding partial data updates.
2. **Consistency**: Ensures that the database moves from one valid state to another. Any transaction that violates data integrity constraints (e.g., issuing more blood than available) will be rejected, maintaining the system's consistency.

3. **Isolation**: Prevents transactions from interfering with each other. In a scenario where multiple staff members are updating the inventory simultaneously, isolation ensures that one transaction's changes are not visible to others until it is complete, preventing data corruption.
4. **Durability**: Once a transaction is committed, its changes are permanent, even in the event of system crashes or power failures. For example, after a donor's blood donation is successfully recorded, the data will persist even if the system experiences a failure immediately afterward.

Concurrency Control:

In a multi-user environment, concurrency control mechanisms are vital to prevent issues like:

- **Lost Updates**: Occur when two transactions overwrite each other's changes, leading to data inconsistency.
- **Temporary Inconsistencies**: When one transaction reads data that is being modified by another transaction.
- **Uncommitted Data**: When a transaction reads data that hasn't been committed yet, leading to inconsistent reads.

Concurrency control techniques include:

1. **Locking Mechanisms**: These ensure that only one transaction can access a particular piece of data at a time. For example, if a blood unit is being issued to a recipient, the system would lock the record to prevent another transaction (like adding a donor's blood) from modifying the same data.
2. **Optimistic Concurrency Control**: Transactions are allowed to execute without locking but before committing, the system checks whether any conflict has occurred. If conflicts are found, the transaction is rolled back and retried.
3. **Timestamp Ordering**: Transactions are given timestamps and the system ensures they are executed in the correct order, preventing older transactions from overwriting newer ones.

These mechanisms ensure that data remains consistent and accurate, even with multiple users interacting with the BBMS at the same time. This is critical to maintaining trust and reliability, especially in environments where accurate information can mean the difference between life and death.

Database System Architectures

For a Blood Bank Management System (BBMS), the choice of database architecture would depend on several factors:

1. **Centralized Database Systems**:

- This could be suitable for a smaller or regional blood bank where data management, security, and maintenance are key. Since blood bank data (like donor information, blood stock levels, and patient requirements) are critical, having a centralized system ensures all data is managed in one place, making it easier to keep track of and maintain.

2. **Distributed Database Systems**:
   - This architecture might be more appropriate for a larger network of blood banks spread across different locations or regions. It would help with reliability and fault tolerance, ensuring that even if one location's database goes down, the others can continue to function. A distributed system would also support more scalability and better performance, especially when handling increasing numbers of donors, requests and data from multiple locations.
3. **Client-Server Architecture**:
   - This is ideal for a web-based BBMS where healthcare staff can access the system remotely to check blood availability, record donations, track inventory, or manage donor data. A client-server architecture allows multiple users to access the system simultaneously while centralizing data management on a server.

For a Blood Bank Management System, a combination of distributed database systems for redundancy and client-server architecture for web access could be the most effective solution to handle scalability and data security needs.

Legal and Ethical Aspects of Database Management

To ensure that the Blood Bank Management System (BBMS) operates ethically and legally, it must incorporate the following practices related to data privacy, data security and ethical considerations:

1. Data Privacy:

- **Sensitive Information Protection:** Personal details, blood donation history, and medical records of both donors and recipients must be safeguarded. This can be achieved through strong encryption and secure storage solutions.
- **Access Control:** Implement role-based access controls (RBAC) to ensure that only authorized personnel can access sensitive data based on their responsibilities. For example, blood bank staff may need to view donor information, but not all staff should have access to medical history.
- **Data Minimization:** Collect and store only the minimum necessary personal data required for blood donation and recipient purposes. This reduces the risk of exposure in case of a breach.
- **Transparency and Consent:** Inform donors and recipients about how their data will be used, and ensure their explicit consent is obtained for storing and processing their data.

2. Data Security:

- **Encryption:** Encrypt sensitive data both **at rest** (when stored) and **in transit** (when being transferred over networks). This ensures that even if unauthorized access occurs, the data remains unreadable.
- **Backup and Recovery:** Regularly back up all data and have a disaster recovery plan in place. This ensures data can be restored quickly and completely in case of corruption or loss.
- **Monitoring and Logging:** Maintain an audit trail of data access and changes to track who accessed the system, when, and what changes were made. This ensures accountability and can help in detecting suspicious activity.
- **Firewalls and Intrusion Detection Systems:** Implement advanced firewalls and intrusion detection/prevention systems to protect the BBMS from external threats such as hacking attempts.

3. Ethical Considerations:

- **Informed Consent:** Ensure that donors and recipients are provided with clear, concise information about how their personal data will be collected, stored, and used. This consent should be documented in compliance with relevant regulations.
- **Anonymization and Pseudonymization:** When possible, anonymize or pseudonymize data to ensure that personal identifiers are removed or masked in datasets used for analysis or reporting, minimizing risks in case of data breaches.
- **Rights of Individuals:** Respect the rights of individuals to access, correct, or delete their data in compliance with privacy laws like GDPR. Donors and recipients should be able to request copies of their data and have inaccurate information corrected or removed.
- **Data Retention Policy:** Establish clear guidelines for how long data will be retained and ensure that data is securely deleted when it is no longer required, in accordance with legal and ethical standards.

4. Compliance with Legal and Regulatory Standards:

- **General Data Protection Regulation (GDPR):** If the BBMS operates within the European Union or processes data from EU citizens, it must comply with GDPR, which mandates strict controls on data processing, including the need for explicit consent, data access rights and data breach notifications.
- **Health Insurance Portability and Accountability Act (HIPAA):** In the U.S., if the BBMS handles health-related data, it must comply with HIPAA regulations to ensure that medical records and other sensitive health data are handled with the highest level of privacy and security.
- **Regular Audits and Assessments:** Conduct regular audits to ensure that all practices related to data privacy, security and ethics are being followed. Also, assess compliance with relevant local or international data protection regulations.

By integrating these measures into the development and operation of the BBMS, we can ensure that it remains legally compliant, secure and ethically responsible in handling sensitive data.

Coursework No. 1: Database Design

Designing the BBMS database.

Problem statement:

Create the database schema using MySQL, including tables like donor_registration, recipient_registration, blood_stock, admin, etc.

Solution:

```
-- phpMyAdmin SQL Dump
-- version 5.2.1
-- https://www.phpmyadmin.net/
--
-- Host: localhost
-- Generation Time: Jan 15, 2025 at 08:07 AM
-- Server version: 10.4.32-MariaDB
-- PHP Version: 8.2.12
SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
--
-- Database: `bbms`
--
-- --------------------------------------------------------
--
-- Table structure for table `admin`
--
CREATE TABLE `admin` (
  `id` int(11) NOT NULL,
  `u_name` varchar(50) NOT NULL,
  `password` varchar(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
--
-- Dumping data for table `admin`
--
INSERT INTO `admin` (`id`, `u_name`, `password`) VALUES
(1, 'admin', '11223344');
-- --------------------------------------------------------
--
-- Table structure for table `blood_stock`
--
CREATE TABLE `blood_stock` (
  `id` int(11) NOT NULL,
  `bgroup` varchar(10) NOT NULL,
  `quantity` int(11) DEFAULT 0
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

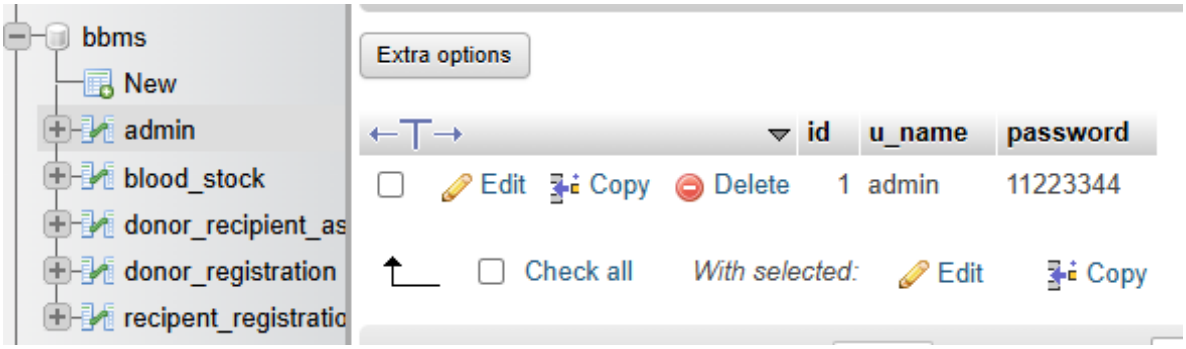
```
--
-- Dumping data for table `blood_stock`
--
INSERT INTO `blood_stock` (`id`, `bgroup`, `quantity`) VALUES
(42, 'A+', 1),
(43, 'A-', 1),
(44, 'B+', 0),
(45, 'B-', 1),
(46, 'O+', 2),
(47, 'O-', 1),
(48, 'AB+', 1),
(49, 'AB-', 1);
-- --------------------------------------------------------
--
-- Table structure for table `donor_recipient_assignment`
--
CREATE TABLE `donor_recipient_assignment` (
  `id` int(11) NOT NULL,
  `donor_id` int(11) NOT NULL,
  `recipient_id` int(11) NOT NULL,
  `assigned_quantity` int(11) NOT NULL,
  `assignment_date` timestamp NOT NULL DEFAULT current_timestamp(),
  `status` enum('pending','completed','canceled') DEFAULT 'pending'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
--
-- Dumping data for table `donor_recipient_assignment`
--
INSERT INTO `donor_recipient_assignment` (`id`, `donor_id`, `recipient_id`,
`assigned_quantity`, `assignment_date`, `status`) VALUES
(58, 67, 41, 1, '2025-01-07 09:14:20', 'completed'),
(59, 75, 41, 1, '2025-01-07 09:15:27', 'canceled'),
(60, 75, 41, 1, '2025-01-07 09:15:57', 'completed');
-- --------------------------------------------------------
--
-- Table structure for table `donor_registration`
--
CREATE TABLE `donor_registration` (
  `id` int(50) NOT NULL,
  `name` varchar(50) DEFAULT NULL,
  `fname` varchar(50) DEFAULT NULL,
  `address` varchar(250) DEFAULT NULL,
  `gender` varchar(50) DEFAULT NULL,
  `age` varchar(25) DEFAULT NULL,
  `bgroup` varchar(20) DEFAULT NULL,
  `email` varchar(200) DEFAULT NULL,
  `pno` varchar(50) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
--
-- Dumping data for table `donor_registration`
--
```

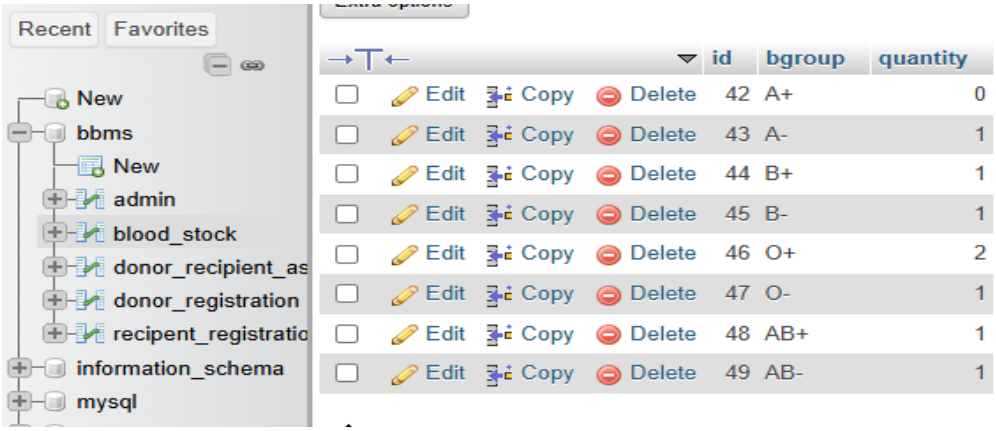
```
INSERT INTO `donor_registration` (`id`, `name`, `fname`, `address`, `gender`, `age`, `bgroup`, `email`, `pno`) VALUES
(67, 'Aminul Islam', 'Abdul Kader', '123, Kazi Nazrul Islam Avenue, Dhaka', 'Male', '30', 'A+', 'aminul.islam@example.com', '0171-1234567'),
(68, 'Fatima Begum', 'Mohammad Ali', '456, Shere-e-Bangla Nagar, Dhaka', 'Female', '25', 'B-', 'fatima.begum@example.com', '0171-2345678'),
(69, 'Rashida Akter', 'Abdul Rahim', '789, Mirpur Road, Dhaka', 'Female', '28', 'O+', 'rashida.akter@example.com', '0171-3456789'),
(70, 'Saidur Rahman', 'Mohammad Shamsul', '101, New Eskaton, Dhaka', 'Male', '35', 'AB+', 'saidur.rahman@example.com', '0171-4567890'),
(71, 'Sharmin Sultana', 'Gazi Mohammad', '202, Uttara, Dhaka', 'Female', '22', 'A-', 'sharmin.sultana@example.com', '0171-5678901'),
(72, 'Tareq Zaman', 'Abdul Gafur', '303, Banani, Dhaka', 'Male', '27', 'O-', 'tareq.zaman@example.com', '0171-6789012'),
(73, 'Khatija Akter', 'Lutfur Rahman', '404, Moghbazar, Dhaka', 'Female', '40', 'B+', 'khatija.akter@example.com', '0171-7890123'),
(74, 'Shamim Hossain', 'Nasir Uddin', '505, Puran Dhaka, Dhaka', 'Male', '31', 'AB-', 'shamim.hossain@example.com', '0171-8901234'),
(75, 'Marium Binte Aslam', 'Sayedul Islam', '606, South Banasree, Dhaka', 'Female', '33', 'A+', 'marium.aslam@example.com', '0171-9012345'),
(76, 'Rifat Ahmed', 'Mohammad Rafique', '707, Shyamoli, Dhaka', 'Male', '24', 'O+', 'rifat.ahmed@example.com', '0171-0123456');
-- --------------------------------------------------------
--
-- Table structure for table `recipent_registration`
--
CREATE TABLE `recipent_registration` (
  `id` int(50) NOT NULL,
  `name` varchar(50) DEFAULT NULL,
  `age` varchar(25) DEFAULT NULL,
  `problems` varchar(200) DEFAULT NULL,
  `bgroup` varchar(50) DEFAULT NULL,
  `quantity` int(25) DEFAULT NULL,
  `place` varchar(250) DEFAULT NULL,
  `time` varchar(50) DEFAULT NULL,
  `pno` varchar(50) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
--
-- Dumping data for table `recipent_registration`
--
INSERT INTO `recipent_registration` (`id`, `name`, `age`, `problems`, `bgroup`, `quantity`, `place`, `time`, `pno`) VALUES
(41, 'Mohammad Ali', '45', 'Anemia', 'A+', 2, 'Dhaka', '2025-01-06', '0181-1234567'),
(42, 'Shama Parveen', '34', 'Heart Disease', 'B-', 3, 'Chittagong', '2025-01-07', '0181-2345678'),
(43, 'Rashidul Islam', '50', 'Cancer', 'O+', 1, 'Rajshahi', '2025-01-08', '0181-3456789'),
(44, 'Fariha Begum', '28', 'Liver Cirrhosis', 'AB+', 2, 'Khulna', '2025-01-09', '0181-4567890'),
(45, 'Mizanur Rahman', '40', 'Kidney Failure', 'A-', 5, 'Sylhet', '2025-01-10', '0181-5678901'),
(46, 'Tariq Zaman', '38', 'Severe Bleeding', 'O-', 1, 'Barisal', '2025-01-11', '0181-6789012'),
```

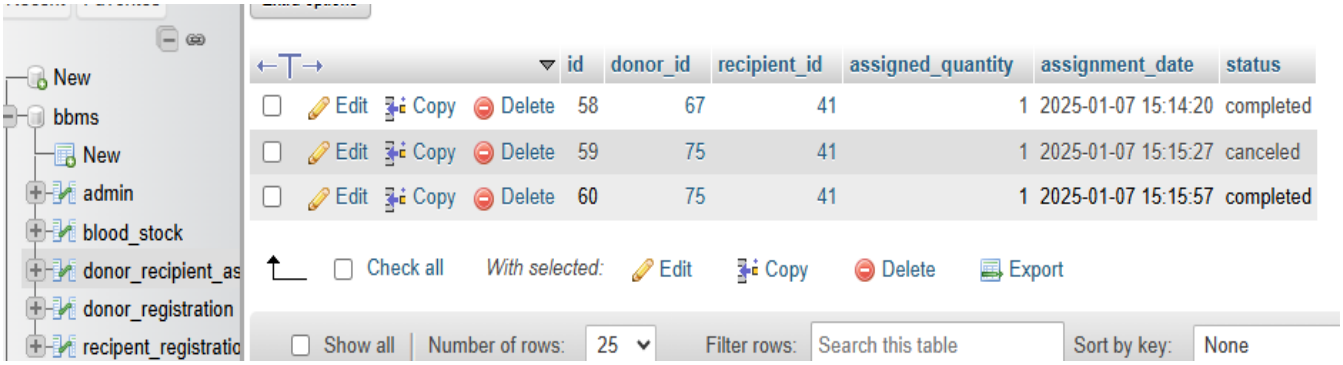
```
(47, 'Sabrina Sultana', '55', 'Tuberculosis', 'B+', 2, 'Mymensingh', '2025-01-12', '0181-7890123'),
(48, 'Shahidul Alam', '60', 'Respiratory Issues', 'AB-', 4, 'Rajbari', '2025-01-13', '0181-8901234'),
(49, 'Nasreen Akter', '31', 'Pneumonia', 'A+', 3, 'Comilla', '2025-01-14', '0181-9012345'),
(50, 'Jahangir Alam', '48', 'Leukemia', 'O+', 2, 'Khulna', '2025-01-15', '0181-0123456');
--
-- Indexes for dumped tables
--
--
-- Indexes for table `admin`
--
ALTER TABLE `admin`
  ADD PRIMARY KEY (`id`);
--
-- Indexes for table `blood_stock`
--
ALTER TABLE `blood_stock`
  ADD PRIMARY KEY (`id`),
  ADD UNIQUE KEY `bgroup` (`bgroup`),
  ADD UNIQUE KEY `unique_bgroup` (`bgroup`);
--
-- Indexes for table `donor_recipient_assignment`
--
ALTER TABLE `donor_recipient_assignment`
  ADD PRIMARY KEY (`id`),
  ADD KEY `donor_id` (`donor_id`),
  ADD KEY `recipient_id` (`recipient_id`);
--
-- Indexes for table `donor_registration`
--
ALTER TABLE `donor_registration`
  ADD PRIMARY KEY (`id`);
--
-- Indexes for table `recipent_registration`
--
ALTER TABLE `recipent_registration`
  ADD PRIMARY KEY (`id`);
--
-- AUTO_INCREMENT for dumped tables
--
--
-- AUTO_INCREMENT for table `admin`
--
ALTER TABLE `admin`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=3;
--
-- AUTO_INCREMENT for table `blood_stock`
--
ALTER TABLE `blood_stock`
```

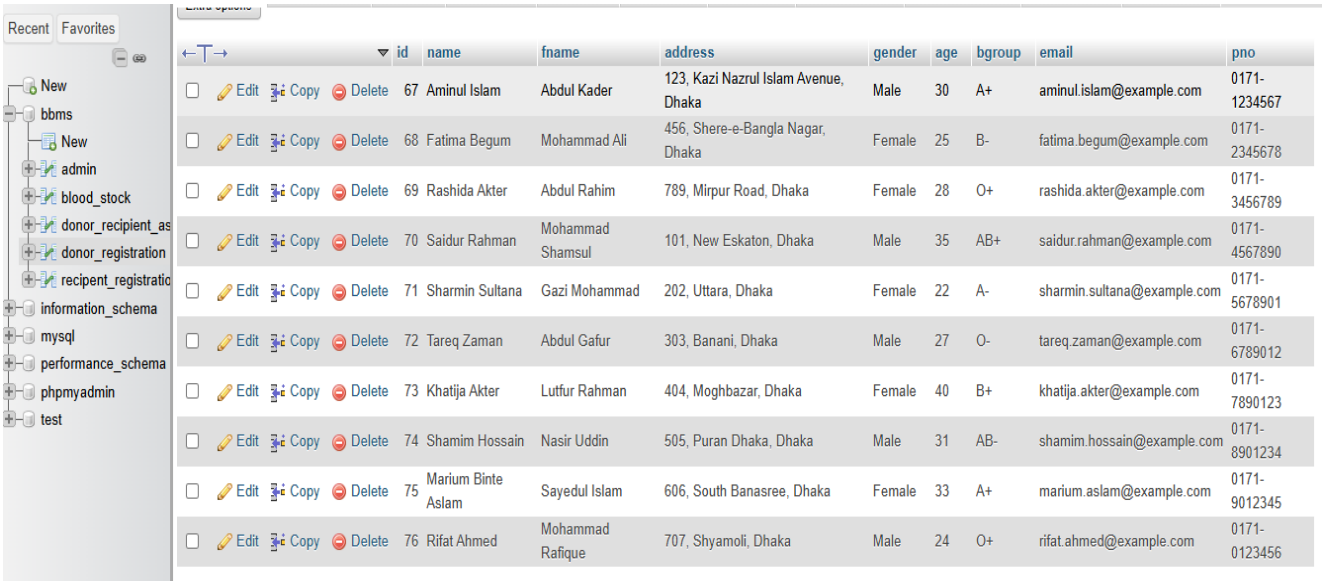
```
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=50;
--
-- AUTO_INCREMENT for table `donor_recipient_assignment`
--
ALTER TABLE `donor_recipient_assignment`
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=61;
--
-- AUTO_INCREMENT for table `donor_registration`
--
ALTER TABLE `donor_registration`
  MODIFY `id` int(50) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=78;
--
-- AUTO_INCREMENT for table `recipent_registration`
--
ALTER TABLE `recipent_registration`
  MODIFY `id` int(50) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=52;
--
-- Constraints for dumped tables
--
--
-- Constraints for table `donor_recipient_assignment`
--
ALTER TABLE `donor_recipient_assignment`
  ADD CONSTRAINT `donor_recipient_assignment_ibfk_1` FOREIGN KEY (`donor_id`)
REFERENCES `donor_registration` (`id`),
  ADD CONSTRAINT `donor_recipient_assignment_ibfk_2` FOREIGN KEY
(`recipient_id`) REFERENCES `recipent_registration` (`id`);
COMMIT;
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
```

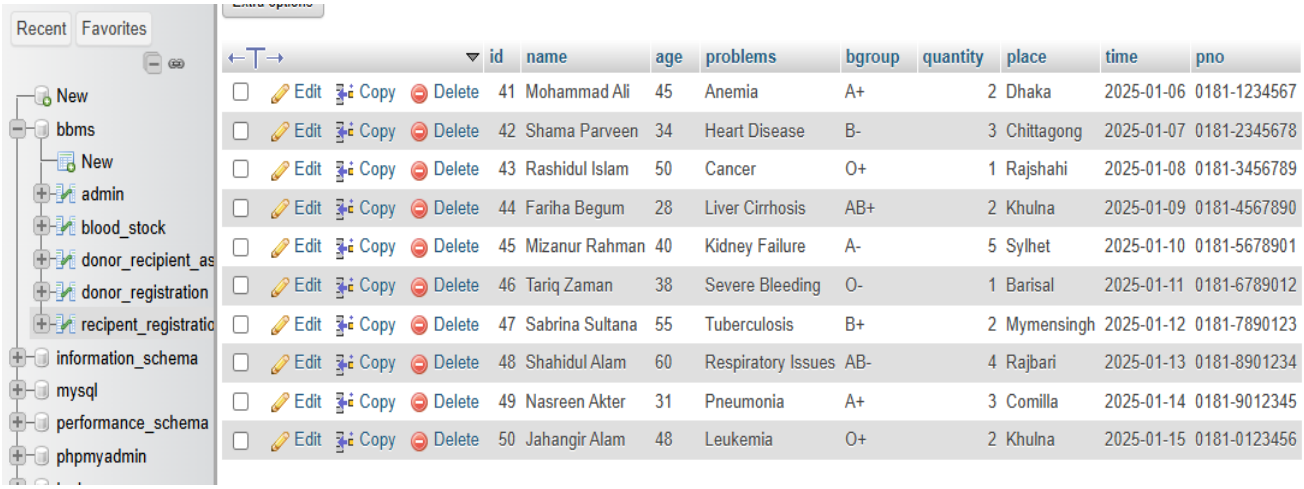
Table picture:




Recent
Favorites
New
bbms
New
admin
blood_stock
donor_recipient_as
donor_registration
recipent_registratio
information_schema
mysql
id
bgroup
quantity
Edit Copy Delete 42 A+ 0
Edit Copy Delete 43 A- 1
Edit Copy Delete 44 B+ 1
Edit Copy Delete 45 B- 1
Edit Copy Delete 46 O+ 2
Edit Copy Delete 47 O- 1
Edit Copy Delete 48 AB+ 1
Edit Copy Delete 49 AB- 1


New
bbms
New
admin
blood_stock
donor_recipient_as
donor_registration
recipent_registratio
id
donor_id
recipient_id
assigned_quantity
assignment_date
status
Edit Copy Delete 58 67 41 1 2025-01-07 15:14:20 completed
Edit Copy Delete 59 75 41 1 2025-01-07 15:15:27 canceled
Edit Copy Delete 60 75 41 1 2025-01-07 15:15:57 completed
Check all
With selected:
Edit
Copy
Delete
Export
Show all
Number of rows:
25
Filter rows:
Search this table
Sort by key:
None


Recent
Favorites
New
bbms
New
admin
blood_stock
donor_recipient_as
donor_registration
recipent_registratio
information_schema
mysql
performance_schema
phpmyadmin
test
id
name
fname
address
gender
age
bgroup
email
pno
Edit Copy Delete 67 Aminul Islam Abdul Kader 123, Kazi Nazrul Islam Avenue, Dhaka Male 30 A+ aminul.islam@example.com 0171-1234567
Edit Copy Delete 68 Fatima Begum Mohammad Ali 456, Shere-e-Bangla Nagar, Dhaka Female 25 B- fatima.begum@example.com 0171-2345678
Edit Copy Delete 69 Rashida Akter Abdul Rahim 789, Mirpur Road, Dhaka Female 28 O+ rashida.akter@example.com 0171-3456789
Edit Copy Delete 70 Saidur Rahman Mohammad Shamsul 101, New Eskaton, Dhaka Male 35 AB+ saidur.rahman@example.com 0171-4567890
Edit Copy Delete 71 Sharmin Sultana Gazi Mohammad 202, Uttara, Dhaka Female 22 A- sharmin.sultana@example.com 0171-5678901
Edit Copy Delete 72 Tareq Zaman Abdul Gafur 303, Banani, Dhaka Male 27 O- tareq.zaman@example.com 0171-6789012
Edit Copy Delete 73 Khatija Akter Lutfur Rahman 404, Moghbazar, Dhaka Female 40 B+ khatija.akter@example.com 0171-7890123
Edit Copy Delete 74 Shamim Hossain Nasir Uddin 505, Puran Dhaka, Dhaka Male 31 AB- shamim.hossain@example.com 0171-8901234
Edit Copy Delete 75 Marium Binte Aslam Sayedul Islam 606, South Banasree, Dhaka Female 33 A+ marium.aslam@example.com 0171-9012345
Edit Copy Delete 76 Rifat Ahmed Mohammad Rafique 707, Shyamoli, Dhaka Male 24 O+ rifat.ahmed@example.com 0171-0123456



	id	name	age	problems	bgroup	quantity	place	time	pno
Edit Copy Delete	41	Mohammad Ali	45	Anemia	A+	2	Dhaka	2025-01-06	0181-1234567
Edit Copy Delete	42	Shama Parveen	34	Heart Disease	B-	3	Chittagong	2025-01-07	0181-2345678
Edit Copy Delete	43	Rashidul Islam	50	Cancer	O+	1	Rajshahi	2025-01-08	0181-3456789
Edit Copy Delete	44	Fariha Begum	28	Liver Cirrhosis	AB+	2	Khulna	2025-01-09	0181-4567890
Edit Copy Delete	45	Mizanur Rahman	40	Kidney Failure	A-	5	Sylhet	2025-01-10	0181-5678901
Edit Copy Delete	46	Tariq Zaman	38	Severe Bleeding	O-	1	Barisal	2025-01-11	0181-6789012
Edit Copy Delete	47	Sabrina Sultana	55	Tuberculosis	B+	2	Mymensingh	2025-01-12	0181-7890123
Edit Copy Delete	48	Shahidul Alam	60	Respiratory Issues	AB-	4	Rajbari	2025-01-13	0181-8901234
Edit Copy Delete	49	Nasreen Akter	31	Pneumonia	A+	3	Comilla	2025-01-14	0181-9012345
Edit Copy Delete	50	Jahangir Alam	48	Leukemia	O+	2	Khulna	2025-01-15	0181-0123456

Coursework No. 2: Admin Login System

Creating a login system for admins.

Problem statement:

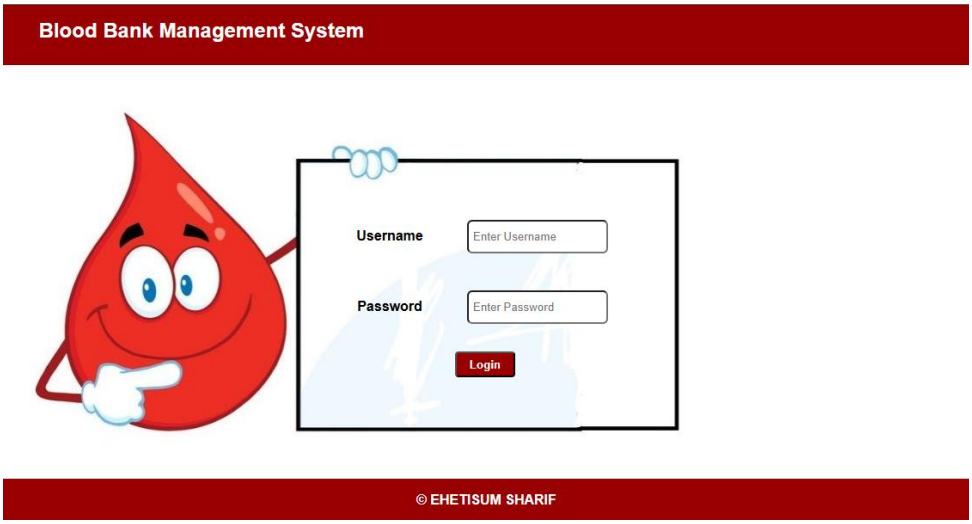
Implement a login page using HTML, CSS, PHP and MySQL for admin authentication.

Solution:

```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
   <title>Admin Login</title>
         <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
   <link rel="stylesheet" type="text/css" href="css/s1.css">
</head>
<body>
   <div id="full">
      <div id="inner_full">
         <div id="header">
            <h2 class="header-title">Blood Bank Management System</h2>
         </div>
         <div id="body1">
            <br><br><br><br>
                                 <form action="" method="post">
               <table align="center" cellpadding="10">
                  <tr>
                     <td width="100px" height="60px"><b>Username</b></td>
                     <td width="200px" height="60px">
                        <input type="text" name="un" placeholder="Enter Username"
```

```
                    style="width: 150px; height: 25px; border-radius: 6px; padding: 5px;">
                </td>
            </tr>
            <tr>
                <td width="100px" height="60px"><b>Password</b></td>
                <td width="200px" height="60px">
                    <input type="password" name="ps" placeholder="Enter Password"
                    style="width: 150px; height: 25px; border-radius: 6px; padding: 5px;">
                </td>
            </tr>
            <tr>
                <td colspan="2" align="center">
                    <input type="submit" name="sub" value="Login"
                    style="width: 70px; height: 30px; border-radius: 4px;
                    background-color: #980002; color: white; font-weight: bold; cursor:
pointer;">
                </td>
            </tr>
        </table>
                                </form>
                                <?php
                                if(isset($_POST['sub']))
                                {
                                        $un=$_POST['un'];
                                        $ps=$_POST['ps'];
                                        $q=$db-> prepare("SELECT* FROM admin
WHERE u_name='$un' AND password='$ps'");
                                        $q-> execute();
                                        $res= $q-> fetchAll(pdo:: FETCH_OBJ);
                                        if($res)
                                        {
                                                $_SESSION['un']=$un;
            header("Location:admin-home.php");
                                        }
                                        else
                                        {
                                                echo "<script>alert('Wrong Username or
Password')</script>";
                                        }
                                }
                                ?>
        </div>
        <div id="footer">
                        <h4 align="center">&copy; EHETISUM SHARIF</h4>
        </div>
      </div>
   </div>
</body>
</html>
```

Screenshots:



Coursework No. 3: Designing the Home Page

Creating the home page for the Blood Bank Management System.

Problem statement:

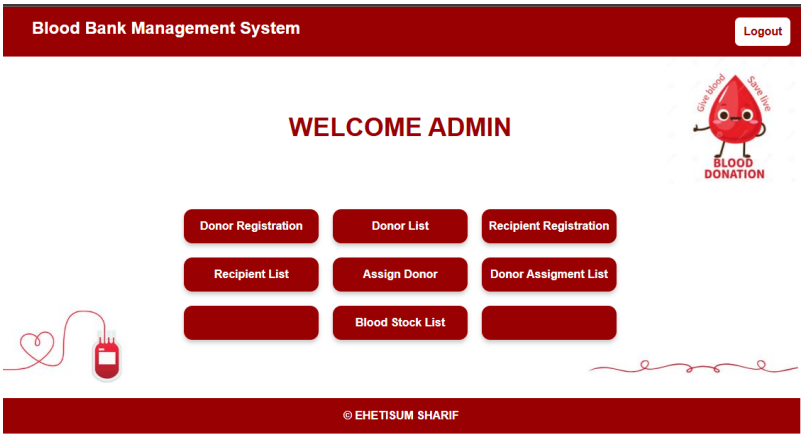
Create an HTML and CSS-based home page with sections for the header (logo, navigation), main content (welcome message, key features), call-to-action buttons (register, login) and footer.

Solution:

```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
   <title>Home</title>
         <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
   <link rel="stylesheet" type="text/css" href="css/s1.css">
</head>
<body>
   <div id="full">
      <div id="inner_full">
         <div id="header">
            <h2 class="header-title">Blood Bank Management System</h2>
           <div class="header-right">
              <a href="logout.php" class="logout-button">Logout</a>
```

```
        </div>
      </div>
      <div id="body">
        <br>
                        <?php
                        $un=$_SESSION['un'];
                        if(!$un)
                        {
                                header("Location:index.php");
                                exit();
                        }
                        ?>
                        <h1 class="welcome-text"> WELCOME ADMIN </h1>
                        <br><br><br>
                         <ul class="menu">
                           <li><a href="Donor_reg.php">Donor Registration</a></li>
          <li><a href="Donor.php">Donor List</a></li>
                                <li><a href="Recipient_reg.php"> Recipient
Registration</a></li>
          <li><a href="Recipient.php">Recipient List</a></li>
                               <li><a href="assign_donor.php">Assign Donor</a></li>
                                <li><a href="assign_list.php">Donor Assigment
List</a></li>
                                <li><a href="#"></a></li>
                                <li><a href="Blood_Stock.php">Blood Stock
List</a></li>
                                <li><a href="#"></a></li>
        </ul>
      </div>
      <div id="footer">
                               <h4 align="center">&copy; EHETISUM SHARIF</h4>
      </div>
    </div>
  </div>
</body>
</html>
```

Screenshots:



Coursework No. 4: PHP Script for Logout

Creating a PHP script for logging out users in the Blood Bank Management System (BBMS).

Problem statement:

The script will destroy the user session and redirect them to the homepage or login page, ensuring users are logged out securely.

Solution:

```
<?php
session_start();
$un= $_SESSION['un'];
session_destroy();
header('Location: index.php');
?>
```

Coursework No. 5: PHP Script for Database Connection

Creating a PHP script to connect to the database in the Blood Bank Management System (BBMS).

Problem statement:

BBMS requires a connection to the MySQL database for managing donor, recipient, and blood stock data. A PHP script should be used to establish this connection.

Solution:

Use PHP's PDO (PHP Data Objects) to create a secure and reusable database connection script with error handling.

```
<?php
$db= new PDO('mysql:host=localhost; dbname=bbms','root','');
?>
```

Coursework No. 6: CSS Script

Topic:
Creating a CSS script for styling the Blood Bank Management System (BBMS) interface.

Problem Statement:

The Blood Bank Management System (BBMS) requires a well-designed, user-friendly interface. A CSS script needs to be created to style the various elements of the BBMS, such as

the navigation bar, forms, buttons, tables and other components, to ensure a professional and consistent look across the application.

Solution:
Use CSS to style the layout, form elements, buttons and tables to enhance the user experience. The styling should focus on usability, readability and visual appeal, with clear differentiation between sections and actions (e.g., login, donor registration, blood stock management).

Code:

```
body, ul, li, h1, h2, p {
   margin: 0;
   padding: 0;
}

#full {
   width: 100%;
   height: auto;
   margin: 0;
   padding: 0;
   font-family: Arial, sans-serif;
   background-color: white;
}
#inner_full {
   width: 80%;
   margin: auto;
   background-color: white;
}
#header {
   display: flex;
   width: 100%;
   height: 70px;
   background-color: #980002;
   color: white;
   line-height: 40px;
   justify-content: space-between;
   padding: 0 20px;
}
#body {
   width: 100%;
   min-height: 430px;
   padding: 15px;
   text-align: center;
   display: flex;
   flex-direction: column;
   justify-content: center;
   align-items: center;
   background: url('../pic/blood c1.jpg');
}
```

```
#body1 {
    width: 100%;
    min-height: 430px;
    padding: 15px;
    text-align: center;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    background: url('../pic/catout.jpg');
}
#footer {
    width: 103%;
    height: 50px;
    background-color: #980002;
    color: white;
    text-align: center;
    line-height: 50px;
}
.welcome-text {
    font-size: 36px;
    font-weight: bold;
    color: #980002;
    margin-bottom: 40px;
}
ul.menu {
    list-style: none;
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
    max-width: 800px;
    margin: 0 auto;
    padding: 0;
}
ul.menu li {
    background-color: #980002;
    color: white;
    border-radius: 12px;
    padding: 15px 10px;
    text-align: center;
    font-size: 16px;
    font-weight: bold;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.2);
    cursor: pointer;
    transition: background-color 0.3s ease-in-out;
}
ul.menu li a {
    text-decoration: none;
    color: white;
    display: block;
```

```
}
ul.menu li:hover {
   background-color: darkred;
}
.header-title {
        padding: 0px 12px;
   margin: 10px;
   font-size: 24px;
   color: white;
}
.header-right {
   display: flex;
   gap: 25px;
   align-items: center;
}
.logout-button {
   padding: 0px 12px;
   background-color: white;
   color: #980002;
   text-decoration: none;
   border-radius: 8px;
   font-weight: bold;
   border: 1px solid #980002;
   transition: 0.3s;
}
.logout-button:hover {
   background-color: #980002;
   color: white;
}
.home {
   padding: 0px 12px;
   background-color: white;
   color: #980002;
   text-decoration: none;
   border-radius: 8px;
   font-weight: bold;
   border: 1px solid #980002;
   transition: 0.3s;
}
.home:hover {
   background-color: #980002;
   color: white;
}
#form{
        width:80%;
        height: 270px;
        background-color: white;
        color: black;
        border-radius: 10px;
```

```
}
#list{
        width: 85%;
        height: 300px;
        background-color: white;
        color: black;
        border-radius: 10px;
        overflow-y: auto;
        text-align: center;
   align-items: center;
}
#ttitle{
        color: #980002;
        font-weight: bold;
}
#st_list {
    width: 30%;
    margin: 0 auto;
    padding: 10px;
    background-color: white;
    border-radius: 10px;
}
.form-container {
   width: 25%;
   margin: auto;
}
.form-container form {
   display: flex;
   flex-direction: column;
}
.form-container label {
   margin: 10px 0 5px;
}
.form-container select, .form-container input {
   padding: 8px;
   margin-bottom: 20px;
   border: 1px solid #980002;
   border-radius: 5px;
}
.form-container button {
   background-color: #980002;
   color: white;
   padding: 10px;
   border: none;
   border-radius: 5px;
   cursor: pointer;
}
.form-container button:hover {
   background-color: darkred;
}
```

```
#alist{
        width: 80%;
        height: 300px;
        background-color: white;
        color: black;
        border-radius: 10px;
        overflow-y: auto;
        text-align: center;
   align-items: center;
}
#alist table td a {
   color: #980002;
   font-weight: bold;
   text-decoration: none;
   padding: 4px 6px;
   background-color: white;
   border: 1px solid #980002;
   border-radius: 5px;
   display: inline-block;
}
#alist table td a:hover {
   background-color: #980002;
   color: white;
   border-color: #980002;
}
#list table td a {
   color: #980002;
   font-weight: bold;
   text-decoration: none;
   padding: 10px 20px;
   background-color: #f5f5f5;
   border: 1px solid #980002;
   border-radius: 4px;
}
#list table td a:hover {
   background-color: #980002;
   color: white; /
   border-color: #980002;
}
  .update-form {
        margin: 20px auto;
     text-align: center;
  }
  .update-form select, .update-form input, .update-form button {
     padding: 3px;
     margin: 5px;
  }
  .update-form button {
     font-size: 14px;
     background-color: #980002;
```

```
    color: white;
    border: none;
    border-radius: 5px;
}
.update-form button:hover {
    background-color: white;
        color: #980002;
}
```

Coursework No. 7: Donor Registration

Creating a Donor Registration Form for the Blood Bank Management System (BBMS).

Problem Statement:
Design a form for donors to register their personal details, such as name, age, blood type and contact information and store the data in the BBMS database.

Solution:

- **HTML Form:** A form to capture donor details (name, age, gender, blood type, contact number).
- **CSS Styling:** Common css for every page.
- **PHP Script:** Handle form submission, validate inputs and insert data into the database.

Code:

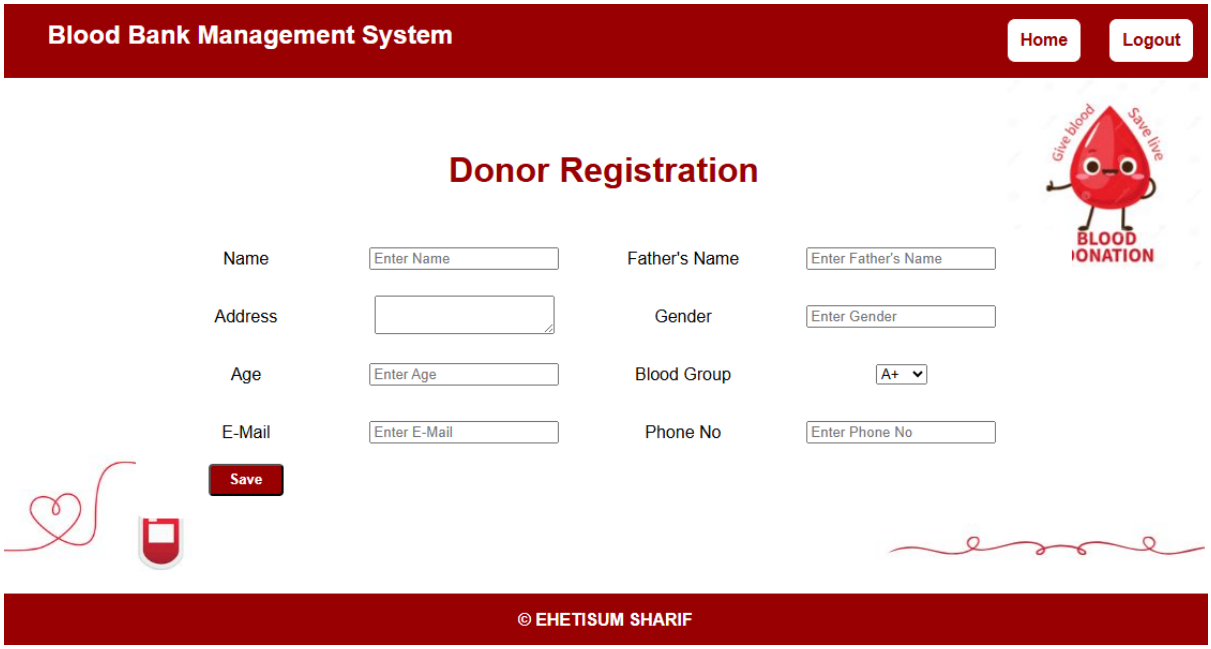
```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
   <title>Donor Registration</title>
   <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
   <link rel="stylesheet" type="text/css" href="css/s1.css">
</head>
<body>
   <div id="full">
      <div id="inner_full">
         <div id="header">
            <h2 class="header-title">Blood Bank Management System</h2>
            <div class="header-right">
               <a href="admin-home.php" class="home">Home</a>
               <a href="logout.php" class="logout-button">Logout</a>
            </div>
         </div>
         <div id="body">
            <br>
            <?php
            $un = $_SESSION['un'];
```

```
            if (!$un) {
                header("Location:index.php");
                exit();
            }
            ?>
            <h1 style="color: #980002;">Donor Registration</h1><br><br>
            <div id="form">
              <form action="" method="post">
              <table>
                <tr>
                  <td width="200px" height="50px">Name</td>
                  <td width="200px" height="50px"><input type="text" name="name"
placeholder="Enter Name"></td>
                  <td width="200px" height="50px">Father's Name</td>
                  <td width="200px" height="50px"><input type="text" name="fname"
placeholder="Enter Father's Name"></td>
                </tr>
                <tr>
                  <td width="200px" height="50px">Address</td>
                  <td width="200px" height="50px"><textarea
name="address"></textarea></td>
                  <td width="200px" height="50px">Gender</td>
                  <td width="200px" height="50px"><input type="text" name="gender"
placeholder="Enter Gender"></td>
                </tr>
                <tr>
                  <td width="200px" height="50px">Age</td>
                  <td width="200px" height="50px"><input type="text" name="age"
placeholder="Enter Age"></td>
                  <td width="200px" height="50px">Blood Group</td>
                  <td width="200px" height="50px">
                    <select name="bgroup">
                        <option> A+ </option>
                        <option> A- </option>
                        <option> B+ </option>
                        <option> B- </option>
                        <option> O+ </option>
                        <option> O- </option>
                        <option> AB+ </option>
                        <option> AB- </option>
                    </select>
                  </td>
                </tr>
                 <tr>
                  <td width="200px" height="50px">E-Mail</td>
                  <td width="200px" height="50px"><input type="text" name="email"
placeholder="Enter E-Mail"></td>
                  <td width="200px" height="50px">Phone No</td>
                  <td width="200px" height="50px"><input type="text" name="pno"
placeholder="Enter Phone No"></td>
```

```
                </tr>
                <tr>
                  <td><input type="submit" name="sub" value="Save"  style="width: 70px; height: 30px; border-radius: 4px; background-color: #980002; color: white; font-weight: bold; cursor: pointer;"></td>
                </tr>
              </table>
              </form>
              <?php
               if (isset($_POST['sub'])) {
                  $name = $_POST['name'];
                  $fname = $_POST['fname'];
                  $address = $_POST['address'];
                  $gender = $_POST['gender'];
                  $age = $_POST['age'];
                  $bgroup = $_POST['bgroup'];
                  $email = $_POST['email'];
                  $pno = $_POST['pno'];

                  $q = $db->prepare("INSERT INTO donor_registration(name, fname, address, gender, age, bgroup, email, pno) VALUES(:name, :fname, :address, :gender, :age, :bgroup, :email, :pno)");
                  $q->bindValue(':name', $name);
                  $q->bindValue(':fname', $fname);
                  $q->bindValue(':address', $address);
                  $q->bindValue(':gender', $gender);
                  $q->bindValue(':age', $age);
                  $q->bindValue(':bgroup', $bgroup);
                  $q->bindValue(':email', $email);
                  $q->bindValue(':pno', $pno);

                  if ($q->execute()) {
                     $stockQuery = $db->prepare("SELECT * FROM blood_stock WHERE bgroup = :bgroup");
                     $stockQuery->bindValue(':bgroup', $bgroup);
                     $stockQuery->execute();

                     if ($stockQuery->rowCount() > 0) {
                        $updateStock = $db->prepare("UPDATE blood_stock SET quantity = quantity + 1 WHERE bgroup = :bgroup");
                        $updateStock->bindValue(':bgroup', $bgroup);
                        $updateStock->execute();
                     } else {
                        $insertStock = $db->prepare("INSERT INTO blood_stock (bgroup, quantity) VALUES (:bgroup, 1)");
                        $insertStock->bindValue(':bgroup', $bgroup);
                        $insertStock->execute();
                     }
                     echo "<script>alert('Donor Registration Successful')</script>";
                  } else {
```

```
                    echo "<script>alert('Error in Registration')</script>";
                }
            }
            ?>
        </div>
    </div>
    <div id="footer">
        <h4 align="center">&copy; EHETISUM SHARIF</h4>
    </div>
  </div>
</body>
</html>
```

Screenshots:



Coursework No. 8: Donor List

Displaying a List of Registered Donors in the Blood Bank Management System (BBMS).

Problem Statement:
Once donors are registered, it is essential to display a list of all registered donors to admins or authorized users. The system should retrieve and display donor details from the database in a readable format.

Solution:
Create a PHP script to fetch all donor records from the database and display them in a table. The table will include details like name, age, gender, blood type and contact number. The list will be presented with a simple design for easy navigation.

Code:

```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
   <title>Donor</title>
   <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
   <link rel="stylesheet" type="text/css" href="css/s1.css">
   <style type="text/css">
      td {
         width: 200px;
         height: 30px;
      }
   </style>
</head>
<body>
   <div id="full">
      <div id="inner_full">
         <div id="header">
            <h2 class="header-title">Blood Bank Management System</h2>
            <div class="header-right">
               <a href="admin-home.php" class="home">Home</a>
               <a href="logout.php" class="logout-button">Logout</a>
            </div>
         </div>
         <div id="body">
            <br>
            <?php
            $un = $_SESSION['un'];
            if (!$un) {
               header("Location:index.php");
               exit();
            }
            ?>
            <h1 style="color: #980002;">Donor List</h1><br>
            <div id="list">
               <table>
                  <tr id="ttitle">
                     <td>Name</td>
                     <td>Father's Name</td>
                     <td>Address</td>
                     <td>Gender</td>
                     <td>Age</td>
                     <td>Blood Group</td>
                     <td>E-Mail</td>
                     <td>Phone no</td>
```

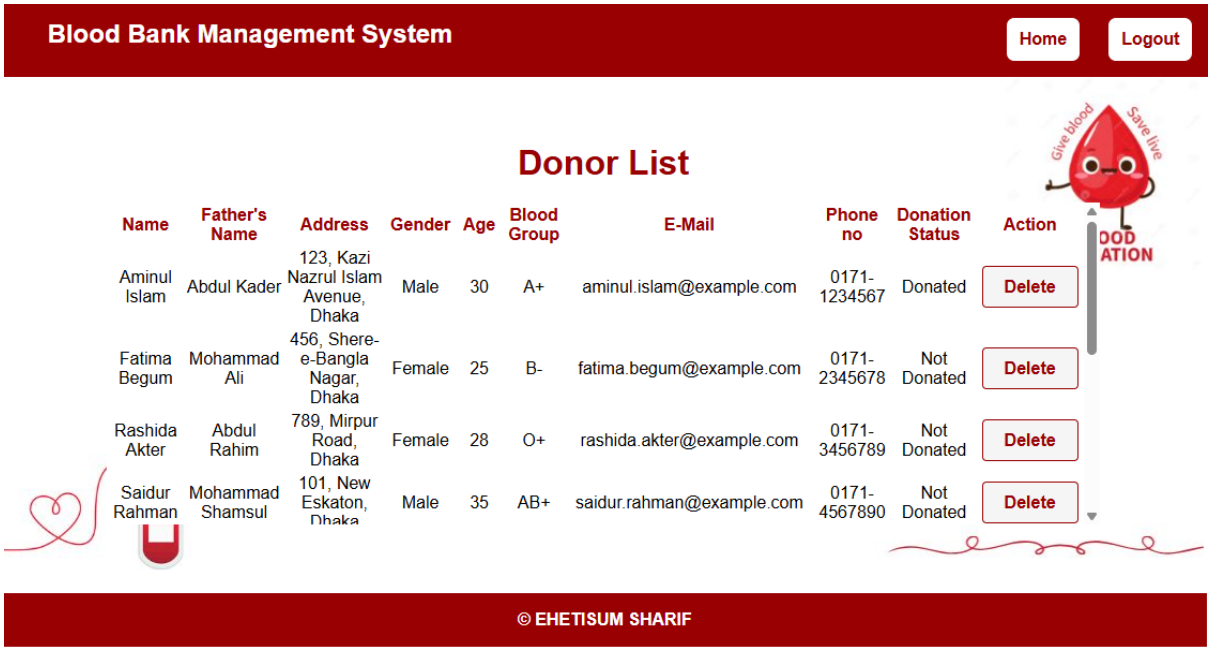
```
                    <td>Donation Status</td>
                    <td>Action</td>
                </tr>
                <?php
                try {
                    $q = $db->query("SELECT donor_registration.*,
                                (CASE
                                    WHEN EXISTS (SELECT 1 FROM donor_recipient_assignment WHERE donor_recipient_assignment.donor_id = donor_registration.id AND donor_recipient_assignment.status = 'completed')
                                    THEN 'Donated'
                                    ELSE 'Not Donated'
                                END) AS donation_status
                                FROM donor_registration");
                    while ($r1 = $q->fetch(PDO::FETCH_OBJ)) {
                        // Check donation status and update blood stock if needed
                        if ($r1->donation_status == 'Donated') {
                            // Remove blood from stock (example: decrease the count of blood for the donor's blood group)
                            $updateStock = $db->prepare("UPDATE blood_stock SET quantity = quantity - 1 WHERE bgroup = ?");
                            $updateStock->execute([$r1->bgroup]);
                        }
                        echo "<tr>
                            <td>{$r1->name}</td>
                            <td>{$r1->fname}</td>
                            <td>{$r1->address}</td>
                            <td>{$r1->gender}</td>
                            <td>{$r1->age}</td>
                            <td>{$r1->bgroup}</td>
                            <td>{$r1->email}</td>
                            <td>{$r1->pno}</td>
                            <td>{$r1->donation_status}</td>
                            <td><a href='delete_donor.php?id={$r1->id}' onclick='return confirm(\"Are you sure you want to delete this donor?\");'>Delete</a></td>
                        </tr>";
                    }
                } catch (PDOException $e) {
                    echo "<tr><td colspan='10'>Error: " . $e->getMessage() . "</td></tr>";
                }
                ?>
            </table>
        </div>
    </div>
    <div id="footer">
        <h4 align="center">&copy; EHETISUM SHARIF</h4>
    </div>
  </div>
 </div>
</body>
```

```
</html>
```

Code for delete action:

```
<?php
include('connection.php');
if (isset($_GET['id'])) {
    $id = $_GET['id'];
    try {
        $db->beginTransaction();

        $stmt1 = $db->prepare("DELETE FROM donor_recipient_assignment WHERE donor_id = :id");
        $stmt1->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt1->execute();

        $stmt2 = $db->prepare("DELETE FROM donor_registration WHERE id = :id");
        $stmt2->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt2->execute();
        $db->commit();

        header("Location: Donor.php");
        exit();
    } catch (PDOException $e) {
        $db->rollBack();
        echo "Error: " . $e->getMessage();
    }
}
?>
```

Screenshots:



Coursework No. 9: Recipient Registration

Creating a Recipient Registration Form for the Blood Bank Management System (BBMS).

Problem Statement:
The BBMS needs a system to register recipients who require blood donations. This form should capture details such as the recipient's name, age, blood type and reason for blood request and store this information in the database.

Solution:

- **HTML Form:** Create a form for recipients to provide their details, such as name, age, blood type and contact information.
- **CSS Styling:** Common css for every page.
- **PHP Script:** Handle form submission, validate the inputs and insert the data into the recipient database.

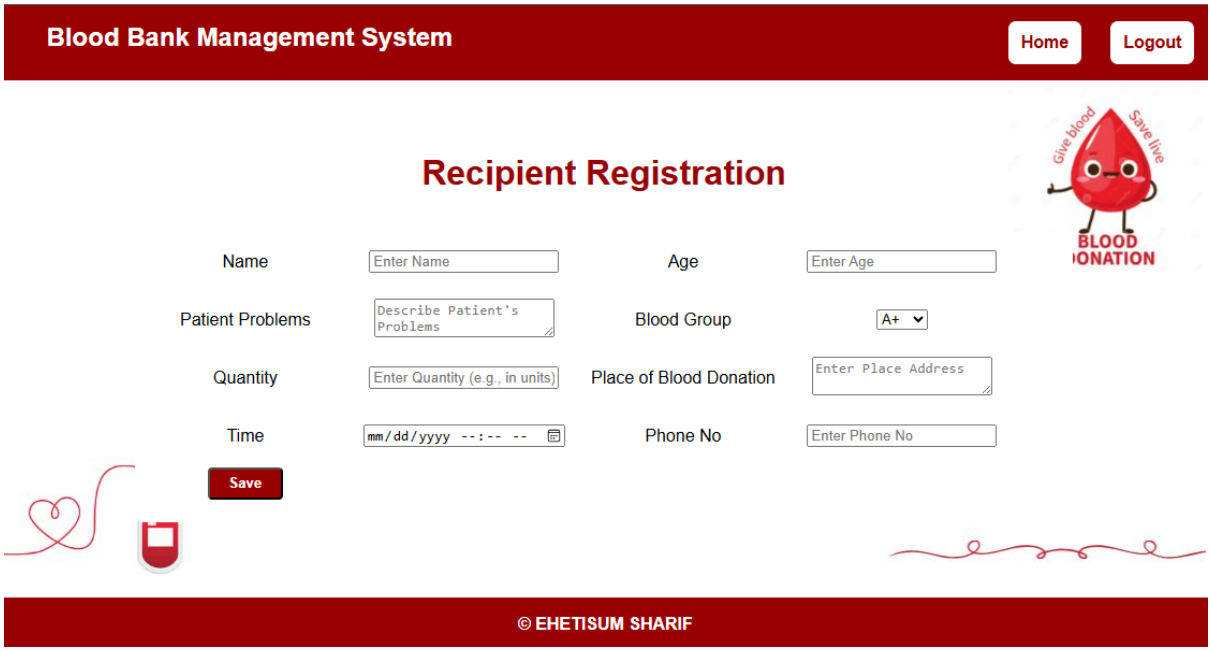
Code:

```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
   <title>Recipient Registration</title>
        <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
   <link rel="stylesheet" type="text/css" href="css/s1.css">
</head>
<body>
   <div id="full">
      <div id="inner_full">
         <div id="header">
            <h2 class="header-title">Blood Bank Management System</h2>
            <div class="header-right">
               <a href="admin-home.php" class="home">Home</a>
               <a href="logout.php" class="logout-button">Logout</a>
            </div>
         </div>
         <div id="body">
            <br>
                                 <?php
                                 $un=$_SESSION['un'];
                                 if(!$un)
                                 {
                                          header("Location:index.php");
                                          exit();
                                 }
```

```
                        ?>
                        <h1 style="color: #980002;">Recipient Registration</h1><br><br>
                        <div id="form">
                          <form action="" method="post">
                          <table>
                            <tr>
                <td width="200px" height="50px">Name</td>
                <td width="200px" height="50px"><input type="text" name="name" placeholder="Enter Name"></td>
                <td width="200px" height="50px">Age</td>
                <td width="200px" height="50px"><input type="text" name="age" placeholder="Enter Age"></td>
              </tr>
              <tr>
                <td width="200px" height="50px">Patient Problems</td>
                <td width="200px" height="50px"><textarea name="problems" placeholder="Describe Patient's Problems"></textarea></td>
                <td width="200px" height="50px">Blood Group</td>
                <td width="200px" height="50px">
                  <select name="bgroup">
                      <option> A+ </option>
                      <option> A- </option>
                      <option> B+ </option>
                      <option> B- </option>
                      <option> O+ </option>
                      <option> O- </option>
                      <option> AB+ </option>
                      <option> AB- </option>
                  </select>
                </td>
              </tr>
              <tr>
                <td width="200px" height="50px">Quantity</td>
                <td width="200px" height="50px"><input type="text" name="quantity" placeholder="Enter Quantity (e.g., in units)"></td>
                <td width="200px" height="50px">Place of Blood Donation</td>
                <td width="200px" height="50px"><textarea name="place" placeholder="Enter Place Address"></textarea></td>
              </tr>
              <tr>
                <td width="200px" height="50px">Time</td>
                <td width="200px" height="50px"><input type="datetime-local" name="time"></td>
                <td width="200px" height="50px">Phone No</td>
                <td width="200px" height="50px"><input type="text" name="pno" placeholder="Enter Phone No"></td>
              </tr>
              <tr>
```

```
                <td><input type="submit" name="sub" value="Save" style="width: 70px;
height: 30px; border-radius: 4px;
                    background-color: #980002; color: white; font-weight: bold; cursor:
pointer;"></td>
                </tr>
                                    </table>
                                    </form>
                                    <?php
            if (isset($_POST['sub'])) {
                $name = $_POST['name'];
                $age = $_POST['age'];
                $problems = $_POST['problems'];
                $bgroup = $_POST['bgroup'];
                $quantity = $_POST['quantity'];
                $place = $_POST['place'];
                $time = $_POST['time'];
                $pno = $_POST['pno'];

                $q = $db->prepare("INSERT INTO recipent_registration(name, age, problems,
bgroup, quantity, place, time, pno)
                    VALUES(:name, :age, :problems, :bgroup, :quantity, :place, :time, :pno)");
                $q->bindValue('name', $name);
                $q->bindValue('age', $age);
                $q->bindValue('problems', $problems);
                $q->bindValue('bgroup', $bgroup);
                $q->bindValue('quantity', $quantity);
                $q->bindValue('place', $place);
                $q->bindValue('time', $time);
                $q->bindValue('pno', $pno);

                if ($q->execute()) {
                    echo "<script>alert('Recipient Registration Successful')</script>";
                } else {
                    echo "<script>alert('Recipient Registration Failed')</script>";
                }
            }
            ?>
          </div>
        </div>
        <div id="footer">
                                    <h4 align="center">&copy; EHETISUM SHARIF</h4>
        </div>
      </div>
    </div>
</body>
</html>
```

Screenshots:



Coursework No. 10: Recipient List

Displaying a List of Registered Recipients in the Blood Bank Management System (BBMS).

Problem Statement:
After recipients are registered, the Blood Bank Management System (BBMS) needs a mechanism to display a list of all registered recipients. The system should allow administrators to view details of recipients, including their name, blood type, contact information and the reason for the blood request.

Solution:
Create a PHP script to fetch all recipient records from the database and display them in a table. The table will include details such as name, age, gender, blood type, contact number and the reason for the blood request. The list will be presented with a simple design for easy navigation.

Code:

```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
   <title>Recipient</title>
   <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
   <link rel="stylesheet" type="text/css" href="css/s1.css">
   <style type="text/css">
      td {
```

```
            width: 200px;
            height: 30px;
        }
    </style>
</head>
<body>
    <div id="full">
        <div id="inner_full">
            <div id="header">
                <h2 class="header-title">Blood Bank Management System</h2>
                <div class="header-right">
                    <a href="admin-home.php" class="home">Home</a>
                    <a href="logout.php" class="logout-button">Logout</a>
                </div>
            </div>
            <div id="body">
                <br>
                <?php
                $un = $_SESSION['un'];
                if (!$un) {
                    header("Location:index.php");
                    exit();
                }
                ?>
                <h1 style="color: #980002;">Recipient List</h1><br>
                <div id="list">
                  <table>
                   <tr id="ttitle">
                      <td>Name</td>
                      <td>Age</td>
                      <td>Patient Problems</td>
                      <td>Blood Group</td>
                      <td>Required Quantity</td>
                      <td>Blood Received</td>
                      <td>Status</td>
                      <td>Place of Blood Donation</td>
                      <td>Time</td>
                      <td>Phone No</td>
                      <td>Action</td>
                   </tr>
                   <?php
                   try {
                      $q = $db->query("
                         SELECT rr.*,
                            COALESCE(SUM(dra.assigned_quantity), 0) AS blood_received
                         FROM recipent_registration rr
                         LEFT JOIN donor_recipient_assignment dra
                            ON rr.id = dra.recipient_id AND dra.status = 'completed'
                         GROUP BY rr.id
                      ");
```

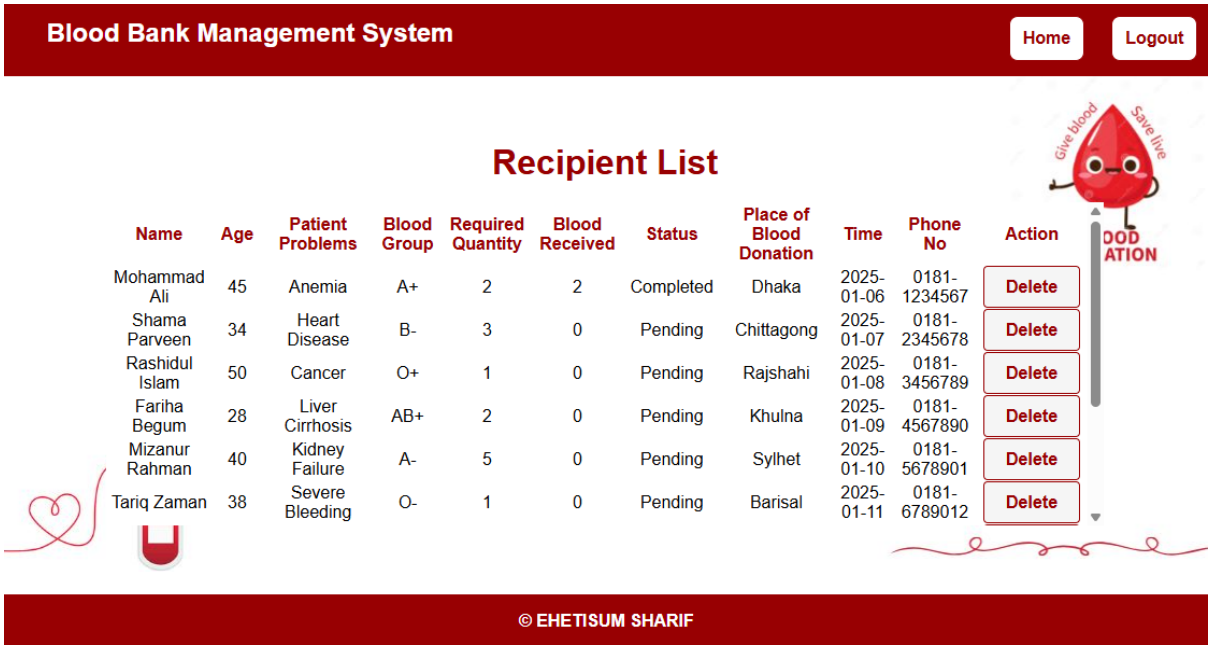
```
                while ($r1 = $q->fetch(PDO::FETCH_OBJ)) {

                    $status = ($r1->blood_received >= $r1->quantity) ? 'Completed' : 'Pending';

                    echo "<tr>
                        <td>{$r1->name}</td>
                        <td>{$r1->age}</td>
                        <td>{$r1->problems}</td>
                        <td>{$r1->bgroup}</td>
                        <td>{$r1->quantity}</td>
                        <td>{$r1->blood_received}</td>
                        <td>{$status}</td>
                        <td>{$r1->place}</td>
                        <td>{$r1->time}</td>
                        <td>{$r1->pno}</td>
                        <td><a href='delete_recipient.php?id={$r1->id}' onclick='return confirm(\"Are you sure you want to delete this recipient?\");'>Delete</a></td>
                    </tr>";
                }
            } catch (PDOException $e) {
                echo "<tr><td colspan='11'>Error: " . $e->getMessage() . "</td></tr>";
            }
            ?>
        </table>
        </div>
    </div>
    <div id="footer">
        <h4 align="center">&copy; EHETISUM SHARIF</h4>
    </div>
  </div>
</body>
</html>
```

Code for delete action:

```
<?php
include('connection.php');
if (isset($_GET['id'])) {
    $id = $_GET['id'];
    try {
        $db->beginTransaction();

        $stmt1 = $db->prepare("DELETE FROM donor_recipient_assignment WHERE recipient_id = :id");
        $stmt1->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt1->execute();

        $stmt2 = $db->prepare("DELETE FROM recipent_registration WHERE id = :id");
        $stmt2->bindParam(':id', $id, PDO::PARAM_INT);
```

```
        $stmt2->execute();
        $db->commit();

        header("Location: Recipient.php");
        exit();
    } catch (PDOException $e) {
        $db->rollBack();
        echo "Error: " . $e->getMessage();
    }
}
?>
```

Screenshots:



Coursework No. 11: Assign Donor

Assigning a Donor to a Blood Recipient.

Problem Statement:
In the Blood Bank Management System, an administrator needs to assign a suitable donor to a recipient's blood request, ensuring the blood type matches and the blood stock is updated accordingly.

Solution:

- Fetch available donors and recipients with pending requests.
- Create a form for the administrator to select a donor and recipient.
- Update the recipient's status to Assigned and reduce the corresponding blood stock.
- Ensure the blood donation process is tracked and managed efficiently.

Code:

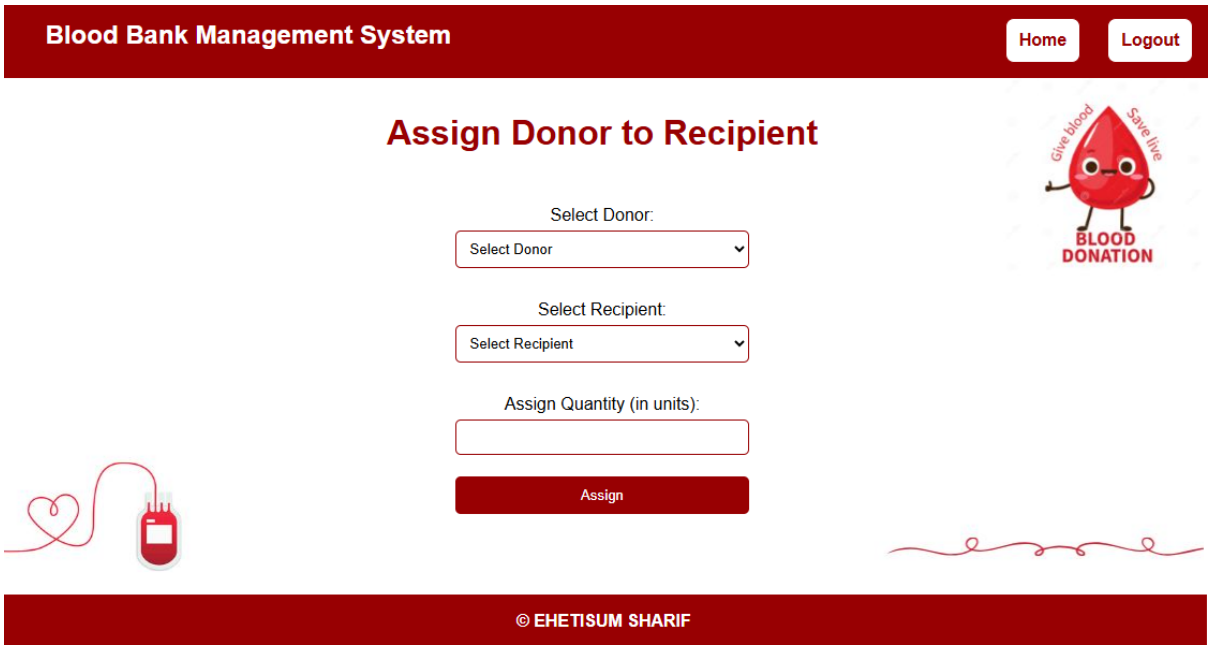
```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
   <title>Assign Donor</title>
   <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
   <link rel="stylesheet" type="text/css" href="css/s1.css">
   <style type="text/css">
      td {
         width: 200px;
         height: 30px;
         text-align: center;
      }
   </style>
</head>
<body>
   <div id="full">
      <div id="inner_full">
         <div id="header">
            <h2 class="header-title">Blood Bank Management System</h2>
            <div class="header-right">
               <a href="admin-home.php" class="home">Home</a>
               <a href="logout.php" class="logout-button">Logout</a>
            </div>
         </div>
         <div id="body">
            <br>
            <?php
            $un = $_SESSION['un'];
            if (!$un) {
               header("Location:index.php");
               exit();
            }
            ?>
            <h1 style="text-align: center; color: #980002;">Assign Donor to Recipient</h1>
            <div class="form-container">
               <form action="assign_donor.php" method="post">
                  <label for="donor_id">Select Donor:</label>
                  <select name="donor_id" required>
                     <option value="">Select Donor</option>
```

```
                    <?php
                    // Include donors with canceled assignments
                    $donors = $db->query(
                        "SELECT * FROM donor_registration
                        WHERE id NOT IN (
                            SELECT donor_id FROM donor_recipient_assignment WHERE status
= 'completed'
                        )"
                    );
                    while ($donor = $donors->fetch(PDO::FETCH_OBJ)) {
                        echo "<option value='{$donor->id}'>{$donor->name} ({$donor-
>bgroup})</option>";
                    }
                    ?>
                </select>

                <label for="recipient_id">Select Recipient:</label>
                <select name="recipient_id" required>
                    <option value="">Select Recipient</option>
                    <?php
                    // Include recipients with canceled assignments
                    $recipients = $db->query(
                        "SELECT rr.*
                         FROM recipent_registration rr
                         LEFT JOIN (
                            SELECT recipient_id, SUM(assigned_quantity) AS total_assigned
                            FROM donor_recipient_assignment
                            WHERE status IN ('pending', 'completed')
                            GROUP BY recipient_id
                         ) dra ON rr.id = dra.recipient_id
                         WHERE dra.total_assigned IS NULL OR dra.total_assigned <
rr.quantity"
                    );
                    while ($recipient = $recipients->fetch(PDO::FETCH_OBJ)) {
                        echo "<option value='{$recipient->id}'>{$recipient->name} ({$recipient-
>bgroup})</option>";
                    }
                    ?>
                </select>

                <label for="quantity">Assign Quantity (in units):</label>
                <input type="number" name="quantity" min="1" required>

                <button type="submit" name="assign">Assign</button>
```

```
</form>

<?php
if (isset($_POST['assign'])) {
    $donor_id = $_POST['donor_id'];
    $recipient_id = $_POST['recipient_id'];
    $quantity = $_POST['quantity'];

    $donor = $db->query("SELECT * FROM donor_registration WHERE id = $donor_id")->fetch(PDO::FETCH_OBJ);
                $recipient = $db->query("SELECT * FROM recipent_registration WHERE id = $recipient_id")->fetch(PDO::FETCH_OBJ);

                if (!$donor || !$recipient) {
                    echo "<p style='color: red; text-align: center;'>Invalid donor or recipient selection.</p>";
                } elseif ($donor->bgroup == $recipient->bgroup) {
                    // Store the donor's blood group to use in the stock update
                    $bgroup = $donor->bgroup;

                    $stmt = $db->prepare("INSERT INTO donor_recipient_assignment (donor_id, recipient_id, assigned_quantity, status)
                                          VALUES (:donor_id, :recipient_id, :quantity, 'pending')");
                    $stmt->bindParam(':donor_id', $donor_id);
                    $stmt->bindParam(':recipient_id', $recipient_id);
                    $stmt->bindParam(':quantity', $quantity);

                    if ($stmt->execute()) {
                        // Now the blood group is correctly passed in the update query
                        $updateStock = $db->prepare("UPDATE blood_stock SET quantity = quantity + :quantity WHERE bgroup = :bgroup");
                        $updateStock->bindValue(':quantity', $quantity, PDO::PARAM_INT);
                        $updateStock->bindValue(':bgroup', $bgroup, PDO::PARAM_STR);
                        $updateStock->execute();
```

```
                                    echo "<script>alert('Donor Assignment Successful');</script>";
                                } else {
                                    echo "<script>alert('Donor Assignment Failed');</script>";
                                }
                            } else {
                                echo "<p style='color: red; text-align: center;'>Blood group mismatch between donor and recipient.</p>";
                            }
                        }
                        ?>
                    </div>
                </div>
                <div id="footer">
                    <h4 align="center">&copy; EHETISUM SHARIF</h4>
                </div>
            </div>
        </div>
    </body>
</html>
```

Screenshots:



Coursework No. 12: Assignment List

Displaying an Assignment List of Donors and Recipients in the Blood Bank Management System (BBMS).

Problem Statement:
The system needs a feature to display a list of all assigned donors and their corresponding recipients. This will help administrators track which donors are fulfilling blood requests and ensure that all assignments are properly recorded.

Solution:
Create a PHP script to retrieve a list of all donor-recipient assignments from the database. Display this data in a table, showing details like donor name, recipient name, blood type and assignment date.

Code:

```
<?php
include('connection.php');
session_start();
?>
<!DOCTYPE html>
<html>
<head>
    <title>Donor Assignment List</title>
    <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
    <link rel="stylesheet" type="text/css" href="css/s1.css">
    <style type="text/css">
        td {
            width: 200px;
            height: 30px;
        }
    </style>
</head>
<body>
    <div id="full">
        <div id="inner_full">
            <div id="header">
                <h2 class="header-title">Blood Bank Management System</h2>
                <div class="header-right">
                    <a href="admin-home.php" class="home">Home</a>
                    <a href="logout.php" class="logout-button">Logout</a>
                </div>
            </div>
            <div id="body">
```

```
<br>
<?php
$un = $_SESSION['un'];
if (!$un) {
   header("Location:index.php");
   exit();
}
?>
<h1 style="color: #980002;">Donor Assignment List</h1><br>
<div id="alist">
   <table>
      <tr id="ttitle">
         <td>Donor Name</td>
         <td>Recipient Name</td>
         <td>Blood Group</td>
         <td>Assigned Quantity</td>
         <td>Status</td>
         <td>Action</td>
      </tr>
      <?php
      try {
         $q = $db->query("SELECT donor.name AS donor_name, recipient.name
AS recipient_name, recipient.bgroup, donor_recipient_assignment.assigned_quantity,
donor_recipient_assignment.id, donor_recipient_assignment.status
                  FROM donor_recipient_assignment
                  JOIN donor_registration AS donor ON donor.id =
donor_recipient_assignment.donor_id
                  JOIN recipent_registration AS recipient ON recipient.id =
donor_recipient_assignment.recipient_id");
         while ($row = $q->fetch(PDO::FETCH_OBJ)) {
            echo "<tr>
               <td>{$row->donor_name}</td>
               <td>{$row->recipient_name}</td>
               <td>{$row->bgroup}</td>
               <td>{$row->assigned_quantity}</td>
               <td>{$row->status}</td>";

            if ($row->status == 'pending') {
               echo "<td>
                     <a href='assign_list.php?action=done&id={$row->id}'>Done</a>
|
                     <a href='assign_list.php?action=cancel&id={$row-
>id}&bgroup={$row->bgroup}&quantity={$row->assigned_quantity}'>Cancel</a>
                  </td>";
```

```
                    }

                    echo "<td><a href='delete_assign_list.php?id={$row->id}'
onclick='return confirm(\"Are you sure you want to delete this
assignment?\");'>Delete</a></td>";
                    echo "</tr>";
                }
            } catch (PDOException $e) {
                echo "<tr><td colspan='6'>Error: " . $e->getMessage() . "</td></tr>";
            }
            ?>
        </table>
    </div>
</div>
<div id="footer">
    <h4 align="center">&copy; EHETISUM SHARIF</h4>
</div>
</div>
</div>

<?php
// Handle Done and Cancel actions
if (isset($_GET['action'])) {
    $action = $_GET['action'];
    $assignment_id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);

    if (!$assignment_id) {
        echo "<script>alert('Invalid assignment ID.');
window.location='assign_list.php';</script>";
        exit();
    }

    try {
        $db->beginTransaction();

        if ($action == 'done') {
            // Mark assignment as completed
            $stmt = $db->prepare("UPDATE donor_recipient_assignment SET status =
'completed' WHERE id = :id");
            $stmt->bindParam(':id', $assignment_id);
            $stmt->execute();
            $db->commit();
```

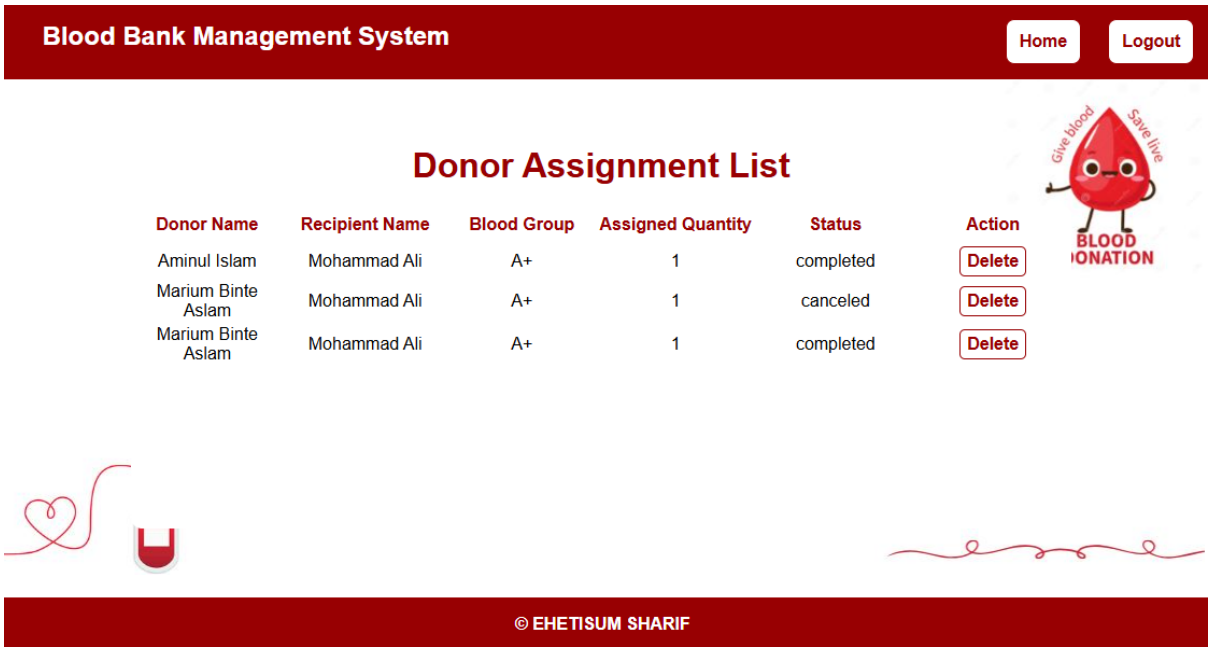
```
            echo "<script>alert('Assignment marked as completed.');
window.location='assign_list.php';</script>";

        } elseif ($action == 'cancel') {
            $bgroup = filter_input(INPUT_GET, 'bgroup', FILTER_SANITIZE_STRING);
            $quantity = filter_input(INPUT_GET, 'quantity', FILTER_VALIDATE_INT);

            if (!$bgroup || !$quantity) {
                echo "<script>alert('Invalid blood group or quantity.');
window.location='assign_list.php';</script>";
                exit();
            }

            // Mark assignment as canceled
            $stmt = $db->prepare("UPDATE donor_recipient_assignment SET status =
'canceled' WHERE id = :id");
            $stmt->bindParam(':id', $assignment_id);
            $stmt->execute();

            // Restock the blood group
           $updateStock = $db->prepare("UPDATE blood_stock SET quantity = quantity +
:quantity WHERE bgroup = :bgroup");
                            $updateStock->bindValue(':quantity', $quantity,
PDO::PARAM_INT);
                            $updateStock->bindValue(':bgroup', $bgroup, PDO::PARAM_STR);
                            $updateStock->execute();



            $db->commit();

            echo "<script>alert('Assignment canceled and blood restocked.');
window.location='assign_list.php';</script>";

        } else {
            $db->rollBack();
            echo "<script>alert('Invalid action.'); window.location='assign_list.php';</script>";
        }

    } catch (Exception $e) {
        $db->rollBack();
        echo "<script>alert('Error: " . $e->getMessage() . "');
window.location='assign_list.php';</script>";
    }
}
```

```
    ?>
</body>
</html>
```

Code for delete action:

```
<?php
include('connection.php');
if (isset($_GET['id'])) {
    $id = $_GET['id'];
    try {
        // Prepare the delete query
        $stmt = $db->prepare("DELETE FROM donor_recipient_assignment WHERE id =
:id");
        $stmt->bindParam(':id', $id, PDO::PARAM_INT);
        $stmt->execute();

        // Redirect back to the assignment list after deletion
        header("Location: assign_list.php");
        exit();
    } catch (PDOException $e) {
        echo "Error: " . $e->getMessage();
    }
}
?>
```

Screenshots:



Coursework No. 13: Blood Stock List

Displaying the Blood Stock List in the Blood Bank Management System (BBMS).

Problem Statement:
In the Blood Bank Management System, it is essential for administrators to view the current stock levels of each blood type to ensure they can manage requests and donations effectively. The system needs to provide a way to display the blood stock list with updated quantities.

Solution:

- Create a PHP script to retrieve blood stock details from the database.
- List all available blood types along with their respective quantities in the database.
- Allow the system to update the stock whenever blood is donated or assigned to a recipient.

Code:

```
<?php
include('connection.php');
session_start();

// Handle form submission
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['bgroup'], $_POST['quantity'], $_POST['action'])) {
    $bgroup = $_POST['bgroup'];
    $quantity = (int) $_POST['quantity'];
    $action = $_POST['action'];

    // Check if quantity is negative
    if ($quantity < 0) {
        echo "<script>alert('Quantity cannot be negative!');</script>";
    } else {
        try {
            $query = $db->prepare("SELECT quantity FROM blood_stock WHERE bgroup = :bgroup");
            $query->bindParam(':bgroup', $bgroup);
            $query->execute();
            $stock = $query->fetch(PDO::FETCH_OBJ);

            if ($stock) {
                if ($action == 'add') {
                    // Add to stock
                    $newQuantity = $stock->quantity + $quantity;
                } elseif ($action == 'reduce') {
                    // Reduce from stock, but ensure quantity doesn't go negative
                    $newQuantity = max(0, $stock->quantity - $quantity);
                } else {
                    throw new Exception("Invalid action specified.");
                }
```

```
            // Update the stock based on the action
            $updateStock = $db->prepare("UPDATE blood_stock SET quantity = :quantity
WHERE bgroup = :bgroup");
            $updateStock->bindValue(':quantity', $newQuantity, PDO::PARAM_INT);
            $updateStock->bindValue(':bgroup', $bgroup, PDO::PARAM_STR);
            $updateStock->execute();
        } else {
            echo "<script>alert('Invalid blood group selected!');</script>";
        }
    } catch (PDOException $e) {
        echo "<script>alert('Error: " . $e->getMessage() . "');</script>";
    }
  }
}
?>

<!DOCTYPE html>
<html>
<head>
  <title>Blood Stock</title>
  <link rel="icon" type="image/jpeg" href="pic/logo.jpeg">
  <link rel="stylesheet" type="text/css" href="css/s1.css">
  <style type="text/css">
    td {
      width: 200px;
      height: 20px;
      text-align: center;
    }
  </style>
</head>
<body>
  <div id="full">
    <div id="inner_full">
      <div id="header">
        <h2 class="header-title">Blood Bank Management System</h2>
        <div class="header-right">
          <a href="admin-home.php" class="home">Home</a>
          <a href="logout.php" class="logout-button">Logout</a>
        </div>
      </div>
      <div id="body">
        <br>
        <?php
        $un = $_SESSION['un'];
        if (!$un) {
          header("Location:index.php");
          exit();
        }
        ?>
```

```
            <h1 style="color: #980002;">Blood Stock</h1><br>
            <div id="st_list">
              <table>
               <tr id="ttitle">
                  <td>Blood Group</td>
                  <td>Quantity</td>
               </tr>
               <?php
               $bloodGroups = ['A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-'];
               foreach ($bloodGroups as $bgroup) {
                  $stockQuery = $db->prepare("SELECT quantity FROM blood_stock WHERE
bgroup = :bgroup");
                  $stockQuery->bindParam(':bgroup', $bgroup);
                  $stockQuery->execute();
                  $stock = $stockQuery->fetch(PDO::FETCH_OBJ);
                  $quantity = $stock && $stock->quantity > 0 ? $stock->quantity : "Empty";
                  echo "<tr>
                        <td>{$bgroup}</td>
                        <td>{$quantity}</td>
                      </tr>";
               }
               ?>
            </table>
            </div>

            <!-- Common Form for Adding or Removing Stock -->
            <div class="update-form">
               <h2 style="color: #980002;">Update Blood Stock</h2>
               <form method="POST">
                  <label for="bgroup">Select Blood Group:</label>
                  <select name="bgroup" id="bgroup" required>
                     <?php
                     foreach ($bloodGroups as $bgroup) {
                        echo "<option value='{$bgroup}'>{$bgroup}</option>";
                     }
                     ?>
                  </select>

                  <label for="quantity">Quantity:</label>
                  <input type="number" name="quantity" id="quantity" min="1" required>

                  <label for="action">Action:</label>
                  <select name="action" id="action" required>
                     <option value="add">Add</option>
                     <option value="reduce">Remove</option>
                  </select>

                  <button type="submit">Update Stock</button>
               </form>
            </div>
```

```
        </div>
        <div id="footer">
           <h4 align="center">&copy; EHETISUM SHARIF</h4>
        </div>
      </div>
   </div>
</body>
</html>
```

Screenshots: